ECP5 Sensor Logger

Release 1.0.0

HDL Team

Nov 01, 2026

Design

1 ECP5 Sensor Logger — Overview **1**
- 1.1 Instantiation Tree . . . 1
- 1.2 Shared Components . . . 1
- 1.3 Modules . . . 2

2 top **3**
- 2.1 top — Entity . . . 3
  - 2.1.1 Annotated Source . . . 4
- 2.2 top — Block Diagram . . . 10
  - 2.2.1 Ports . . . 11
  - 2.2.2 Generics . . . 11
  - 2.2.3 Signals . . . 12
- 2.3 top — State Machine . . . 12
- 2.4 top — Timing Diagrams . . . 12
- 2.5 Processes . . . 13
  - 2.5.1 p_ovf_sync . . . 13
    - 2.5.1.1 Description . . . 13
    - 2.5.1.2 Source . . . 13
  - 2.5.2 Summary . . . 13
- 2.6 top — Clock Domain Analysis . . . 13
  - 2.6.1 Clock Domain . . . 13
- 2.7 top — Reset Domain Analysis . . . 14
  - 2.7.1 Reset Domain . . . 14
- 2.8 Submodules . . . 14
  - 2.8.1 apb_bridge . . . 14
    - 2.8.1.1 apb_bridge — Entity . . . 14
    - 2.8.1.2 apb_bridge — Block Diagram . . . 17
    - 2.8.1.3 apb_bridge — State Machine . . . 21
    - 2.8.1.4 apb_bridge — Timing Diagrams . . . 21
    - 2.8.1.5 Processes . . . 21
    - 2.8.1.6 apb_bridge — Clock Domain Analysis . . . 22
    - 2.8.1.7 apb_bridge — Reset Domain Analysis . . . 23
  - 2.8.2 axil_adapter . . . 23
    - 2.8.2.1 axil_adapter — Entity . . . 23
    - 2.8.2.2 axil_adapter — Block Diagram . . . 25
    - 2.8.2.3 axil_adapter — State Machine . . . 29
    - 2.8.2.4 axil_adapter — Timing Diagrams . . . 29
    - 2.8.2.5 axil_adapter — Clock Domain Analysis . . . 29
    - 2.8.2.6 axil_adapter — Reset Domain Analysis . . . 29
    - 2.8.2.7 Submodules . . . 29
  - 2.8.3 axis_pipeline . . . 39
    - 2.8.3.1 axis_pipeline — Entity . . . 39

2.8.3.2 axis_pipeline — Block Diagram . . . 40
2.8.3.3 axis_pipeline — State Machine . . . 44
2.8.3.4 axis_pipeline — Timing Diagrams . . . 44
2.8.3.5 axis_pipeline — Clock Domain Analysis . . . 44
2.8.3.6 axis_pipeline — Reset Domain Analysis . . . 44
2.8.3.7 Submodules . . . 44
2.8.4 reset_sync . . . 53
2.8.4.1 reset_sync — Entity . . . 53
2.8.4.2 reset_sync — Block Diagram . . . 56
2.8.4.3 reset_sync — State Machine . . . 59
2.8.4.4 reset_sync — Timing Diagrams . . . 59
2.8.4.5 Processes . . . 59
2.8.4.6 reset_sync — Clock Domain Analysis . . . 59
2.8.4.7 reset_sync — Reset Domain Analysis . . . 60
2.8.5 sample_capture . . . 60
2.8.5.1 sample_capture — Entity . . . 60
2.8.5.2 sample_capture — Block Diagram . . . 63
2.8.5.3 sample_capture — State Machine . . . 67
2.8.5.4 sample_capture — Timing Diagrams . . . 67
2.8.5.5 Processes . . . 67
2.8.5.6 sample_capture — Clock Domain Analysis . . . 69
2.8.5.7 sample_capture — Reset Domain Analysis . . . 71
2.8.5.8 Submodules . . . 71
2.8.6 sample_ctrl . . . 83
2.8.6.1 sample_ctrl — Entity . . . 83
2.8.6.2 sample_ctrl — Block Diagram . . . 87
2.8.6.3 sample_ctrl State Machine . . . 91
2.8.6.4 sample_ctrl — Timing Diagrams . . . 91
2.8.6.5 Processes . . . 92
2.8.6.6 sample_ctrl — Clock Domain Analysis . . . 95
2.8.6.7 sample_ctrl — Reset Domain Analysis . . . 95
2.8.7 sample_stream . . . 96
2.8.7.1 sample_stream — Entity . . . 96
2.8.7.2 sample_stream — Block Diagram . . . 97
2.8.7.3 sample_stream — State Machine . . . 100
2.8.7.4 sample_stream — Timing Diagrams . . . 100
2.8.7.5 sample_stream — Clock Domain Analysis . . . 100
2.8.7.6 sample_stream — Reset Domain Analysis . . . 100
2.8.7.7 Submodules . . . 100

3 Register Maps **101**

4 Documentation Coverage **103**

5 Testbenches **105**
5.1 VUnit . . . 105
5.1.1 tb_sample_ctrl . . . 105
5.1.2 tb_stream_filter . . . 105
5.2 cocotb . . . 106
5.2.1 test_axil_adapter . . . 106
5.3 Directed . . . 106
5.3.1 tb_axil_adapter . . . 106
5.3.2 tb_smoke . . . 107
5.3.3 tb_top . . . 107

6 Build **109**
6.1 Summary . . . 109
6.2 Utilization . . . 109
6.2.1 Device utilisation (nextpnr) . . . 109
6.2.2 By module (Vivado) . . . 109
6.3 Timing — Vivado . . . 110
6.4 Warnings . . . 110

7 Indices **111**

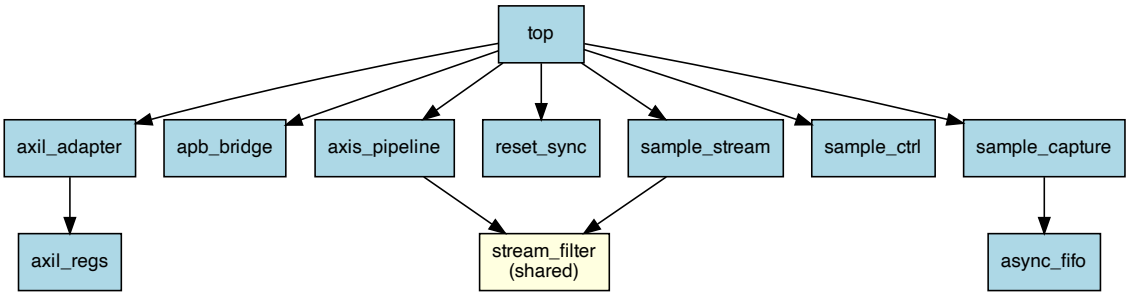
Chapter 1

ECP5 Sensor Logger — Overview

This project contains 11 VHDL/SV modules.

Top-level module: *top*

1.1 Instantiation Tree



1.2 Shared Components

The following modules are instantiated by more than one parent:

- `stream_filter` — instantiated by: `axis_pipeline`, `sample_stream`

1.3 Modules

Module	Source File	Description	Instantiated by	Instantiates
axis_p	axis_pi sv	Filtered pass-through for an external byte stream. Presents flat AXI4-Stream ports, wraps them in interface instances and runs	top	stream_filter
axil_re	axil_re sv	AXI4-Lite register file for the sampler. Five 32-bit registers: control (start, filter enable, clear, clear overflow), a read-only	axil_adapt	—
sample_ctr	sample_ vhd	Sampler controller. Runs one capture window per start request: it arms, enables the capture	top	—
top	top. vhd	Sampler top level. Captures samples from a converter clock into the system clock, runs a capture	*(top)*	apb_bridge, axil_adapter, axis_pipeline, reset_sync, sample_capture, sample_ctrl, sample_stream
stream	stream_ sv	Packet filter for a byte stream. Looks at the first byte of each packet: if it is at or above the threshold	axis_pipel sample_str	—
sample_ca	sample_ sv	Sample capture across a clock boundary. Registers samples from a free-running converter clock (adc_clk) and hands	top	async_fifo
axil_ad	axil_ad sv	Flat AXI4-Lite to register file adapter. Presents the register bus as flat AXI4-Lite subordinate ports (so a VHDL	top	axil_regs
*async_*	async_f vhd	Asynchronous FIFO. Moves words from the write clock domain to the read clock domain. Each side	sample_cap	—
reset_syn	reset_s vhd	Reset synchroniser. Takes an asynchronous active-low reset and delivers it to one clock domain:	top	—
apb_br	apb_bri vhd	APB to AXI4-Lite bridge. Turns each APB transfer into one AXI4-Lite read or write and completes the	top	—
sample_str	sample_ sv	Captured samples as a filtered byte stream. Sends the low byte of each sample as a one-byte packet and runs it through a	top	stream_filter

Chapter 2

top

2.1 top — Entity

Source file: `src/top.vhd`.

ENTITY **`top`** IS

Sampler top level.

Captures samples from a converter clock into the system clock, runs a capture window under register control (APB in, through an AXI4-Lite register file) and sends the samples, and a second external byte stream, through packet filters. The converter and system clocks are asynchronous to each other.

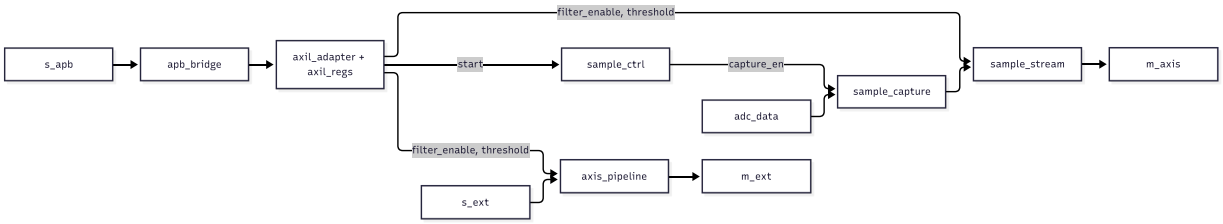


Fig. 1: Top-level structure

`Generics`

Generic	Type	Default	Description
WINDOW_CYCLES	positive	1024	Length of the capture window in system clock cycles.

`Ports`

Port	Type	Mode	Description
clk	std_logic	in	System clock.
arst_n	std_logic	in	System-domain asynchronous active-low reset input.
adc_clk	std_logic	in	Converter clock; asynchronous to clk.
adc_arst_n	std_logic	in	Converter-domain asynchronous active-low reset input.
adc_data	std_logic_vector(11 downto 0)	in	Sample from the converter.
s_apb_psel	std_logic	in	APB subordinate port for register access.
s_apb_penable	std_logic	in	
s_apb_pwrite	std_logic	in	
s_apb_paddr	std_logic_vector(7 downto 0)	in	
s_apb_pwdata	std_logic_vector(31 downto 0)	in	
s_apb_prdata	std_logic_vector(31 downto 0)	out	
s_apb_pready	std_logic	out	
m_axis_tdata	std_logic_vector(7 downto 0)	out	Filtered captured samples (low byte, one-byte packets).
m_axis_tvalid	std_logic	out	
m_axis_tready	std_logic	in	
m_axis_tlast	std_logic	out	
s_ext_tdata	std_logic_vector(7 downto 0)	in	External byte stream in.
s_ext_tvalid	std_logic	in	
s_ext_tready	std_logic	out	
s_ext_tlast	std_logic	in	
m_ext_tdata	std_logic_vector(7 downto 0)	out	External byte stream out, after filtering.
m_ext_tvalid	std_logic	out	
m_ext_tready	std_logic	in	
m_ext_tlast	std_logic	out	
led_busy	std_logic	out	High while a capture is in progress.
led_overflow	std_logic	out	High after a sample was lost to a full FIFO.

2.1.1 Annotated Source

Listing 1: src/top.vhd

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- Sampler top level.
5  --
6  -- Captures samples from a converter clock into the system clock, runs a capture
7  -- window under register control (APB in, through an AXI4-Lite register file)
8  -- and sends the samples, and a second external byte stream, through packet
9  -- filters. The converter and system clocks are asynchronous to each other.
10 --
```

(continues on next page)

(continued from previous page)

```
-- .. mermaid::
--    :caption: Top-level structure
--
--    flowchart LR
--      APB[s_apb] --> BR[apb_bridge] --> RG[axil_adapter + axil_regs]
--      RG -- start --> CT[sample_ctrl]
--      ADC[adc_data] --> CAP[sample_capture]
--      CT -- capture_en --> CAP
--      CAP --> SS[sample_stream] --> MA[m_axis]
--      EXT[s_ext] --> AP[axis_pipeline] --> ME[m_ext]
--      RG -- filter_enable, threshold --> SS
--      RG -- filter_enable, threshold --> AP
entity top is
    generic (
        -- Length of the capture window in system clock cycles.
        WINDOW_CYCLES : positive := 1024
    );
    port (
        -- System clock.
        clk            : in  std_logic;
        -- System-domain asynchronous active-low reset input.
        arst_n         : in  std_logic;
        -- Converter clock; asynchronous to clk.
        adc_clk        : in  std_logic;
        -- Converter-domain asynchronous active-low reset input.
        adc_arst_n     : in  std_logic;
        -- Sample from the converter.
        adc_data       : in  std_logic_vector(11 downto 0);
        -- APB subordinate port for register access.
        s_apb_psel     : in  std_logic;
        s_apb_penable  : in  std_logic;
        s_apb_pwrite   : in  std_logic;
        s_apb_paddr    : in  std_logic_vector(7 downto 0);
        s_apb_pwdata   : in  std_logic_vector(31 downto 0);
        s_apb_prdata   : out std_logic_vector(31 downto 0);
        s_apb_pready   : out std_logic;
        -- Filtered captured samples (low byte, one-byte packets).
        m_axis_tdata   : out std_logic_vector(7 downto 0);
        m_axis_tvalid  : out std_logic;
        m_axis_tready  : in  std_logic;
        m_axis_tlast   : out std_logic;
        -- External byte stream in.
        s_ext_tdata    : in  std_logic_vector(7 downto 0);
        s_ext_tvalid   : in  std_logic;
        s_ext_tready   : out std_logic;
        s_ext_tlast    : in  std_logic;
        -- External byte stream out, after filtering.
        m_ext_tdata    : out std_logic_vector(7 downto 0);
        m_ext_tvalid   : out std_logic;
        m_ext_tready   : in  std_logic;
        m_ext_tlast    : out std_logic;
        -- High while a capture is in progress.
```

(continues on next page)

(continued from previous page)

```
        led_busy       : out std_logic;
        -- High after a sample was lost to a full FIFO.
        led_overflow   : out std_logic
    );
end entity top;

architecture rtl of top is

    -- The components below are SystemVerilog modules; the synthesis flow
    -- resolves them by name. Their widths follow full_pkg::SAMPLE_W (12).
    component sample_capture is
        port (
            clk          : in  std_logic;
            rst_n        : in  std_logic;
            enable       : in  std_logic;
            clear_overflow : in std_logic;
            adc_clk      : in  std_logic;
            adc_rst      : in  std_logic;
            adc_data     : in  std_logic_vector(11 downto 0);
            sample_data  : out std_logic_vector(11 downto 0);
            sample_valid : out std_logic;
            sample_ready : in  std_logic;
            overflow     : out std_logic
        );
    end component sample_capture;

    component sample_stream is
        port (
            clk           : in  std_logic;
            rst           : in  std_logic;
            enable        : in  std_logic;
            threshold     : in  std_logic_vector(7 downto 0);
            sample_data   : in  std_logic_vector(11 downto 0);
            sample_valid  : in  std_logic;
            sample_ready  : out std_logic;
            m_axis_tdata  : out std_logic_vector(7 downto 0);
            m_axis_tvalid : out std_logic;
            m_axis_tready : in  std_logic;
            m_axis_tlast  : out std_logic
        );
    end component sample_stream;

    component axis_pipeline is
        port (
            clk           : in  std_logic;
            rst           : in  std_logic;
            enable        : in  std_logic;
            clear         : in  std_logic;
            threshold     : in  std_logic_vector(7 downto 0);
            s_axis_tdata  : in  std_logic_vector(7 downto 0);
            s_axis_tvalid : in  std_logic;
            s_axis_tready : out std_logic;
```

(continues on next page)

(continued from previous page)

```
            s_axis_tlast  : in  std_logic;
            m_axis_tdata  : out std_logic_vector(7 downto 0);
            m_axis_tvalid : out std_logic;
            m_axis_tready : in  std_logic;
            m_axis_tlast  : out std_logic;
            count         : out std_logic_vector(15 downto 0)
        );
    end component axis_pipeline;

    component axil_adapter is
        port (
            clk           : in  std_logic;
            rst           : in  std_logic;
            s_axil_awaddr  : in  std_logic_vector(7 downto 0);
            s_axil_awvalid : in  std_logic;
            s_axil_awready : out std_logic;
            s_axil_wdata   : in  std_logic_vector(31 downto 0);
            s_axil_wstrb   : in  std_logic_vector(3 downto 0);
            s_axil_wvalid  : in  std_logic;
            s_axil_wready  : out std_logic;
            s_axil_bresp   : out std_logic_vector(1 downto 0);
            s_axil_bvalid  : out std_logic;
            s_axil_bready  : in  std_logic;
            s_axil_araddr  : in  std_logic_vector(7 downto 0);
            s_axil_arvalid : in  std_logic;
            s_axil_arready : out std_logic;
            s_axil_rdata   : out std_logic_vector(31 downto 0);
            s_axil_rresp   : out std_logic_vector(1 downto 0);
            s_axil_rvalid  : out std_logic;
            s_axil_rready  : in  std_logic;
            start          : out std_logic;
            filter_enable  : out std_logic;
            clear          : out std_logic;
            clear_overflow : out std_logic;
            threshold      : out std_logic_vector(7 downto 0);
            state_code     : in  std_logic_vector(1 downto 0);
            overflow       : in  std_logic;
            count          : in  std_logic_vector(15 downto 0)
        );
    end component axil_adapter;

    -- Resets: asserted at once, released on the domain's clock.
    signal srst_n, adc_srst_n : std_logic;
    signal rst, adc_rst       : std_logic;

    -- Register bus between the APB bridge and the register file.
    signal awaddr, araddr   : std_logic_vector(7 downto 0);
    signal wdata, rdata     : std_logic_vector(31 downto 0);
    signal wstrb            : std_logic_vector(3 downto 0);
    signal bresp, rresp     : std_logic_vector(1 downto 0);
    signal awvalid, awready, wvalid, wready, bvalid, bready : std_logic;
    signal arvalid, arready, rvalid, rready                 : std_logic;
```

(continues on next page)

(continued from previous page)

```
167
168     -- Control.
169     signal start, filter_enable, clear, clear_overflow : std_logic;
170     signal threshold                   : std_logic_vector(7 downto 0);
171     signal capture_en, busy            : std_logic;
172     signal state_code                  : std_logic_vector(1 downto 0);
173     signal ext_count                   : std_logic_vector(15 downto 0);
174
175     -- Capture path.
176     signal sample_data  : std_logic_vector(11 downto 0);
177     signal sample_valid, sample_ready : std_logic;
178     signal overflow     : std_logic;
179     -- overflow is set on adc_clk; two flops bring it onto clk.
180     signal ovf_meta, ovf_sync : std_logic := '0';
181
182 begin
183
184     -- System-domain reset.
185     u_rst_sys : entity work.reset_sync
186         port map (clk => clk, arst_n => arst_n, srst_n => srst_n);
187
188     -- Converter-domain reset.
189     u_rst_adc : entity work.reset_sync
190         port map (clk => adc_clk, arst_n => adc_arst_n, srst_n => adc_srst_n);
191
192     rst     <= not srst_n;
193     adc_rst <= not adc_srst_n;
194
195     -- p_ovf_sync: Brings the converter-domain overflow flag onto clk.
196     p_ovf_sync : process(clk) is
197     begin
198         if rising_edge(clk) then
199             ovf_meta <= overflow;
200             ovf_sync <= ovf_meta;
201         end if;
202     end process p_ovf_sync;
203
204     led_busy     <= busy;
205     led_overflow <= ovf_sync;
206
207     -- Register access: APB to AXI4-Lite to the register file.
208     u_apb : entity work.apb_bridge
209         port map (
210             clk => clk, rst => rst,
211             s_apb_psel => s_apb_psel, s_apb_penable => s_apb_penable,
212             s_apb_pwrite => s_apb_pwrite, s_apb_paddr => s_apb_paddr,
213             s_apb_pwdata => s_apb_pwdata, s_apb_prdata => s_apb_prdata,
214             s_apb_pready => s_apb_pready,
215             m_axil_awaddr => awaddr, m_axil_awvalid => awvalid, m_axil_awready =>␣
↪awready,
216             m_axil_wdata => wdata, m_axil_wstrb => wstrb,
217             m_axil_wvalid => wvalid, m_axil_wready => wready,
```

(continues on next page)

(continued from previous page)

```
            m_axil_bresp => bresp, m_axil_bvalid => bvalid, m_axil_bready => bready,
            m_axil_araddr => araddr, m_axil_arvalid => arvalid, m_axil_arready =>␣
↪arready,
            m_axil_rdata => rdata, m_axil_rresp => rresp,
            m_axil_rvalid => rvalid, m_axil_rready => rready
        );

    u_regs : axil_adapter
        port map (
            clk => clk, rst => rst,
            s_axil_awaddr => awaddr, s_axil_awvalid => awvalid, s_axil_awready =>␣
↪awready,
            s_axil_wdata => wdata, s_axil_wstrb => wstrb,
            s_axil_wvalid => wvalid, s_axil_wready => wready,
            s_axil_bresp => bresp, s_axil_bvalid => bvalid, s_axil_bready => bready,
            s_axil_araddr => araddr, s_axil_arvalid => arvalid, s_axil_arready =>␣
↪arready,
            s_axil_rdata => rdata, s_axil_rresp => rresp,
            s_axil_rvalid => rvalid, s_axil_rready => rready,
            start => start, filter_enable => filter_enable, clear => clear,
            clear_overflow => clear_overflow,
            threshold => threshold, state_code => state_code,
            overflow => ovf_sync, count => ext_count
        );

    -- Capture window control.
    u_ctrl : entity work.sample_ctrl
        generic map (RUN_CYCLES => WINDOW_CYCLES)
        port map (
            clk => clk, rst => rst, start => start, overflow => ovf_sync,
            capture_en => capture_en, busy => busy, state_code => state_code
        );

    -- Capture across the clock boundary.
    u_capture : sample_capture
        port map (
            clk => clk, rst_n => srst_n, enable => capture_en, clear_overflow => clear_
↪overflow,
            adc_clk => adc_clk, adc_rst => adc_rst, adc_data => adc_data,
            sample_data => sample_data, sample_valid => sample_valid,
            sample_ready => sample_ready, overflow => overflow
        );

    -- Captured samples through the packet filter.
    u_samples : sample_stream
        port map (
            clk => clk, rst => rst, enable => filter_enable, threshold => threshold,
            sample_data => sample_data, sample_valid => sample_valid,
            sample_ready => sample_ready,
            m_axis_tdata => m_axis_tdata, m_axis_tvalid => m_axis_tvalid,
            m_axis_tready => m_axis_tready, m_axis_tlast => m_axis_tlast
        );
```

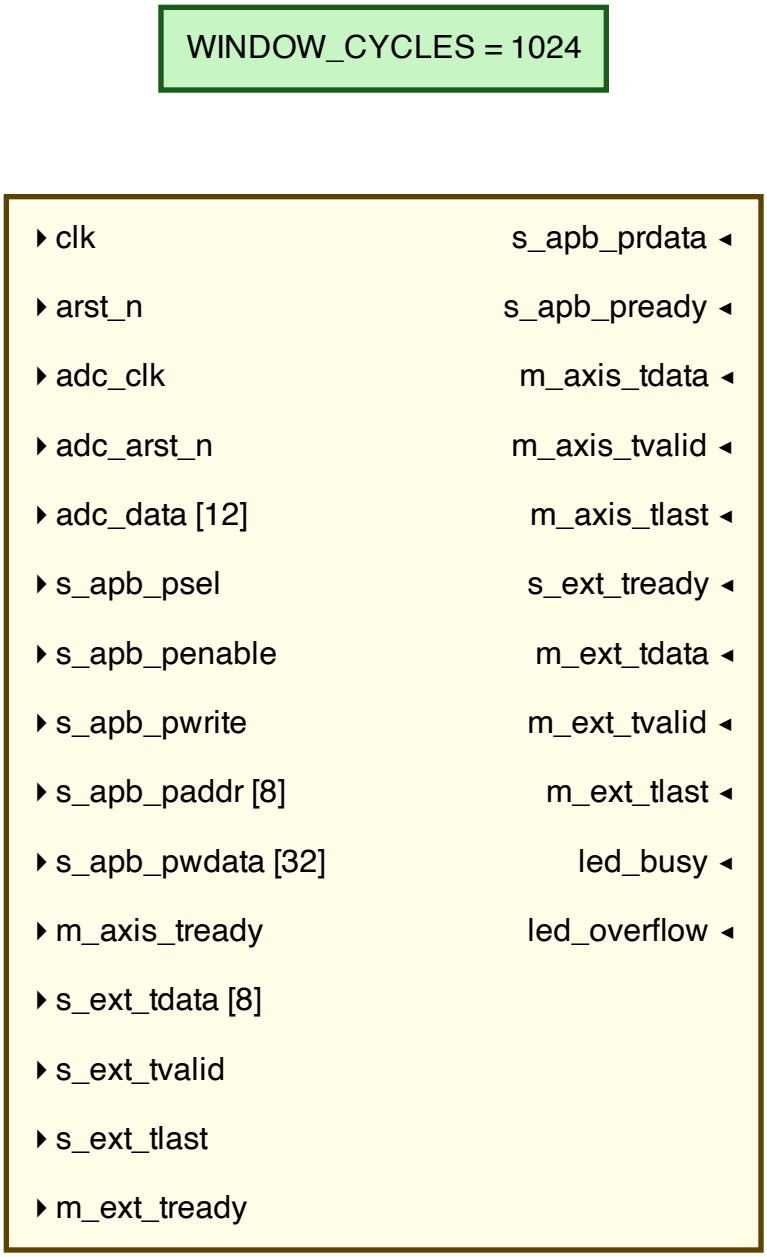
(continues on next page)

(continued from previous page)

```
266
267     -- External stream through the packet filter.
268     u_ext : axis_pipeline
269         port map (
270             clk => clk, rst => rst, enable => filter_enable, clear => clear,
271             threshold => threshold,
272             s_axis_tdata => s_ext_tdata, s_axis_tvalid => s_ext_tvalid,
273             s_axis_tready => s_ext_tready, s_axis_tlast => s_ext_tlast,
274             m_axis_tdata => m_ext_tdata, m_axis_tvalid => m_ext_tvalid,
275             m_axis_tready => m_ext_tready, m_axis_tlast => m_ext_tlast,
276             count => ext_count
277         );
278
279 end architecture rtl;
```

2.2 top — Block Diagram

Auto-extracted from `top.vhd`.



2.2.1 Ports

2.2.2 Generics

Name	Type	Default	Description
`WINDOW_CYCLES`	`positive`	`1024`	Length of the capture window in system clock cycles.

2.2.3 Signals

Signal	Type	Description
srst_n	std_logic	Resets: asserted at once, released on the domain's clock.
adc_srst_n	std_logic	Resets: asserted at once, released on the domain's clock.
rst	std_logic	—
adc_rst	std_logic	—
awaddr	std_logic_vector(7 downto 0)	Register bus between the APB bridge and the register file.
araddr	std_logic_vector(7 downto 0)	Register bus between the APB bridge and the register file.
wdata	std_logic_vector(31 downto 0)	—
rdata	std_logic_vector(31 downto 0)	—
wstrb	std_logic_vector(3 downto 0)	—
bresp	std_logic_vector(1 downto 0)	—
rresp	std_logic_vector(1 downto 0)	—
awvalid	std_logic	—
awready	std_logic	—
wvalid	std_logic	—
wready	std_logic	—
bvalid	std_logic	—
bready	std_logic	—
arvalid	std_logic	—
arready	std_logic	—
rvalid	std_logic	—
rready	std_logic	—
start	std_logic	Control.
filter_enable	std_logic	Control.
clear	std_logic	Control.
clear_overflow	std_logic	Control.
threshold	std_logic_vector(7 downto 0)	—
capture_en	std_logic	—
busy	std_logic	—
state_code	std_logic_vector(1 downto 0)	—
ext_count	std_logic_vector(15 downto 0)	—
sample_data	std_logic_vector(11 downto 0)	Capture path.
sample_valid	std_logic	—
sample_ready	std_logic	—
overflow	std_logic	—
ovf_meta	std_logic	overflow is set on adc_clk; two flops bring it onto clk.
ovf_sync	std_logic	overflow is set on adc_clk; two flops bring it onto clk.

2.3 top — State Machine

No FSM detected in `top.vhd`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.4 top — Timing Diagrams

All timing diagrams extracted from `top.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.5 Processes

Auto-extracted from `top.vhd`.

2.5.1 p_ovf_sync

Source file: `top.vhd`
Source line: 196
Block type: `process`
Sensitivity list: `clk`

2.5.1.1 Description

p_ovf_sync: Brings the converter-domain overflow flag onto clk.

2.5.1.2 Source

```
p_ovf_sync : process(clk) is
begin
    if rising_edge(clk) then
        ovf_meta <= overflow;
        ovf_sync <= ovf_meta;
    end if;
end process p_ovf_sync;
```

2.5.2 Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_ovf_sync	`process`	`clk`	196	—

2.6 top — Clock Domain Analysis

Auto-extracted from `top.vhd`.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

2.6.1 Clock Domain

Clock	Clocked Processes
`clk`	`p_ovf_sync`

2.7 top — Reset Domain Analysis

Auto-extracted from `top.vhd`.

Warning

All clocked processes in this module have **no reset**. Ensure initial state is handled by the synthesis tool or test infrastructure.

2.7.1 Reset Domain

Process	Clock	Reset	Style
`p_ovf_sync`	`clk`	—	—

2.8 Submodules

2.8.1 apb_bridge

2.8.1.1 apb_bridge — Entity

Source file: `src/apb_bridge.vhd`.

ENTITY **`apb_bridge`** IS

APB to AXI4-Lite bridge.

Turns each APB transfer into one AXI4-Lite read or write and completes the APB access when the AXI4-Lite response arrives. One transfer at a time. **`Generics`**

Generic	Type	Default	Description

`Ports`

Port	Type	Mode	Description
clk	std_logic	in	System clock.
rst	std_logic	in	Synchronous active-high reset.
s_apb_psel	std_logic	in	APB subordinate port.
s_apb_penable	std_logic	in	
s_apb_pwrite	std_logic	in	
s_apb_paddr	std_logic_vector(7 downto 0)	in	
s_apb_pwdata	std_logic_vector(31 downto 0)	in	
s_apb_prdata	std_logic_vector(31 downto 0)	out	
s_apb_pready	std_logic	out	
m_axil_awaddr	std_logic_vector(7 downto 0)	out	AXI4-Lite manager port.
m_axil_awvalid	std_logic	out	
m_axil_awready	std_logic	in	
m_axil_wdata	std_logic_vector(31 downto 0)	out	
m_axil_wstrb	std_logic_vector(3 downto 0)	out	
m_axil_wvalid	std_logic	out	
m_axil_wready	std_logic	in	
m_axil_bresp	std_logic_vector(1 downto 0)	in	
m_axil_bvalid	std_logic	in	
m_axil_bready	std_logic	out	
m_axil_araddr	std_logic_vector(7 downto 0)	out	
m_axil_arvalid	std_logic	out	
m_axil_arready	std_logic	in	
m_axil_rdata	std_logic_vector(31 downto 0)	in	
m_axil_rresp	std_logic_vector(1 downto 0)	in	
m_axil_rvalid	std_logic	in	
m_axil_rready	std_logic	out	

Annotated Source

Listing 2: src/apb_bridge.vhd

```
library ieee;
use ieee.std_logic_1164.all;

-- APB to AXI4-Lite bridge.
--
-- Turns each APB transfer into one AXI4-Lite read or write and completes the
-- APB access when the AXI4-Lite response arrives. One transfer at a time.
entity apb_bridge is
    port (
        -- System clock.
        clk           : in  std_logic;
        -- Synchronous active-high reset.
        rst           : in  std_logic;
        -- APB subordinate port.
        s_apb_psel    : in  std_logic;
        s_apb_penable : in  std_logic;
        s_apb_pwrite  : in  std_logic;
        s_apb_paddr   : in  std_logic_vector(7 downto 0);
        s_apb_pwdata  : in  std_logic_vector(31 downto 0);
```

(continues on next page)

(continued from previous page)

```
        s_apb_prdata  : out std_logic_vector(31 downto 0);
        s_apb_pready  : out std_logic;
        -- AXI4-Lite manager port.
        m_axil_awaddr  : out std_logic_vector(7 downto 0);
        m_axil_awvalid : out std_logic;
        m_axil_awready : in  std_logic;
        m_axil_wdata   : out std_logic_vector(31 downto 0);
        m_axil_wstrb   : out std_logic_vector(3 downto 0);
        m_axil_wvalid  : out std_logic;
        m_axil_wready  : in  std_logic;
        m_axil_bresp   : in  std_logic_vector(1 downto 0);
        m_axil_bvalid  : in  std_logic;
        m_axil_bready  : out std_logic;
        m_axil_araddr  : out std_logic_vector(7 downto 0);
        m_axil_arvalid : out std_logic;
        m_axil_arready : in  std_logic;
        m_axil_rdata   : in  std_logic_vector(31 downto 0);
        m_axil_rresp   : in  std_logic_vector(1 downto 0);
        m_axil_rvalid  : in  std_logic;
        m_axil_rready  : out std_logic
    );
end entity apb_bridge;

architecture rtl of apb_bridge is
    type t_phase is (IDLE, WR_ADDR, WR_RESP, RD_ADDR, RD_DATA, FINISH);
    signal phase : t_phase := IDLE;
    signal addr  : std_logic_vector(7 downto 0)  := (others => '0');
    signal wdata : std_logic_vector(31 downto 0) := (others => '0');
    signal rdata : std_logic_vector(31 downto 0) := (others => '0');
begin

    -- p_phase: Walks one APB transfer through the AXI4-Lite channels.
    p_phase : process(clk) is
    begin
        if rising_edge(clk) then
            if rst = '1' then
                phase <= IDLE;
            else
                case phase is
                    when IDLE =>
                        if s_apb_psel = '1' and s_apb_penable = '0' then
                            addr  <= s_apb_paddr;
                            wdata <= s_apb_pwdata;
                            if s_apb_pwrite = '1' then
                                phase <= WR_ADDR;
                            else
                                phase <= RD_ADDR;
                            end if;
                        end if;
                    when WR_ADDR =>
                        if m_axil_awready = '1' then
                            phase <= WR_RESP;
```

(continues on next page)

(continued from previous page)

```
                        end if;
                    when WR_RESP =>
                        if m_axil_bvalid = '1' then
                            phase <= FINISH;
                        end if;
                    when RD_ADDR =>
                        if m_axil_arready = '1' then
                            phase <= RD_DATA;
                        end if;
                    when RD_DATA =>
                        if m_axil_rvalid = '1' then
                            rdata <= m_axil_rdata;
                            phase <= FINISH;
                        end if;
                    when FINISH =>
                        if s_apb_penable = '1' then
                            phase <= IDLE;
                        end if;
                end case;
            end if;
        end if;
    end process p_phase;

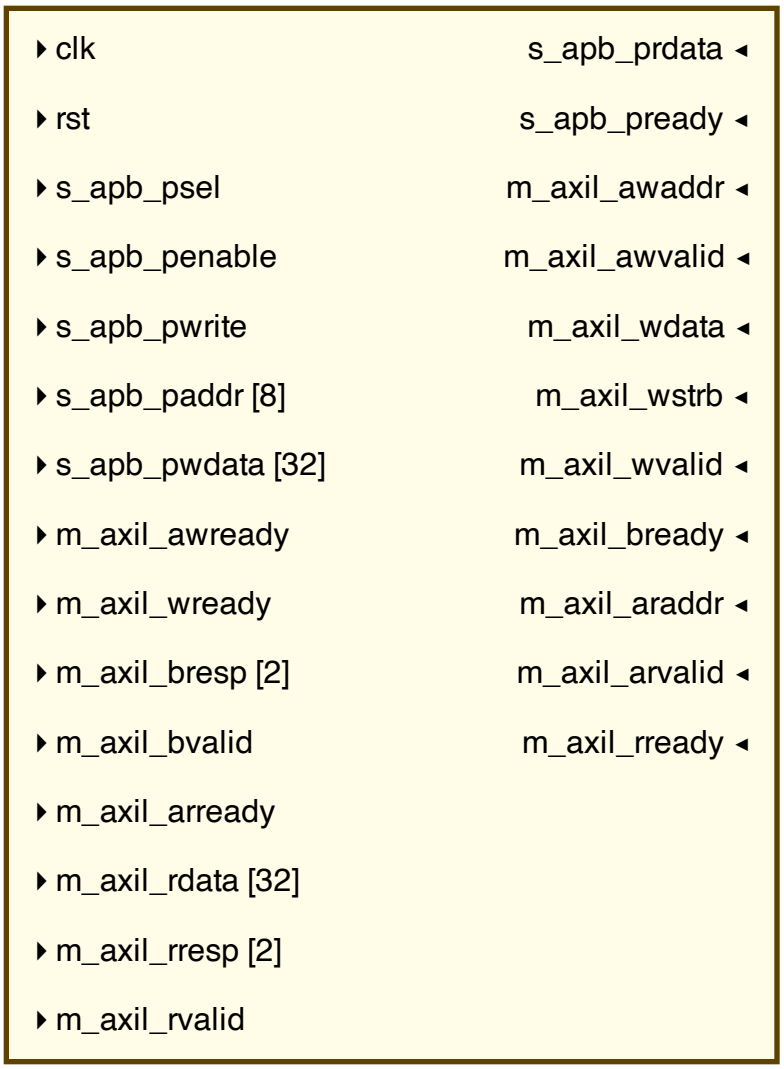
    m_axil_awaddr  <= addr;
    m_axil_awvalid <= '1' when phase = WR_ADDR else '0';
    m_axil_wdata   <= wdata;
    m_axil_wstrb   <= "1111";
    m_axil_wvalid  <= '1' when phase = WR_ADDR else '0';
    m_axil_bready  <= '1' when phase = WR_RESP else '0';
    m_axil_araddr  <= addr;
    m_axil_arvalid <= '1' when phase = RD_ADDR else '0';
    m_axil_rready  <= '1' when phase = RD_DATA else '0';

    s_apb_prdata <= rdata;
    s_apb_pready <= '1' when phase = FINISH else '0';

end architecture rtl;
```

2.8.1.2 apb_bridge — Block Diagram

Auto-extracted from `apb_bridge.vhd`.



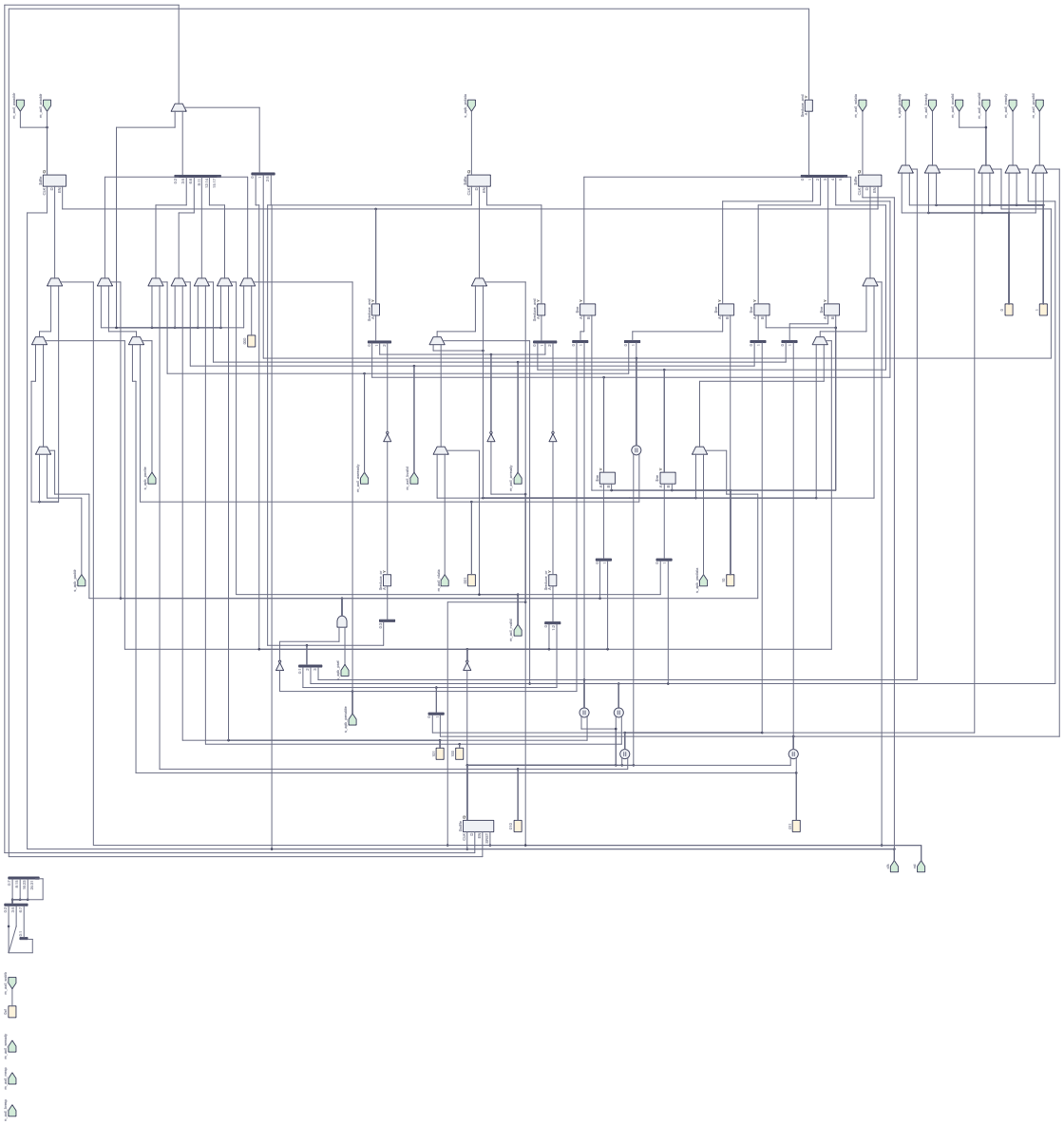
Ports

Signals

Signal	Type	Description
phase	t_phase	—
addr	std_logic_vector(7 downto 0)	—
wdata	std_logic_vector(31 downto 0)	—
rdata	std_logic_vector(31 downto 0)	—

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.



2.8.1.3 apb_bridge — State Machine

No FSM detected in `apb_bridge.vhd`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.8.1.4 apb_bridge — Timing Diagrams

All timing diagrams extracted from `apb_bridge.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.8.1.5 Processes

Auto-extracted from `apb_bridge.vhd`.

p_phase

Source file: `apb_bridge.vhd`
Source line: 52
Block type: `process`
Sensitivity list: `clk`

Description

p_phase: Walks one APB transfer through the AXI4-Lite channels.

Signal Assignments

State	addr	wdata	phase	rdata
IDLE	s_apb_paddr	s_apb_pwdata	RD_ADDR	—
WR_ADDR	—	—	WR_RESP	—
WR_RESP	—	—	FINISH	—
RD_ADDR	—	—	RD_DATA	—
RD_DATA	—	—	FINISH	m_axil_rdata
FINISH	—	—	IDLE	—

Source

```
p_phase : process(clk) is
begin
    if rising_edge(clk) then
        if rst = '1' then
            phase <= IDLE;
        else
            case phase is
                when IDLE =>
                    if s_apb_psel = '1' and s_apb_penable = '0' then
```

(continues on next page)

(continued from previous page)

```
                        addr  <= s_apb_paddr;
                        wdata <= s_apb_pwdata;
                        if s_apb_pwrite = '1' then
                            phase <= WR_ADDR;
                        else
                            phase <= RD_ADDR;
                        end if;
                    end if;
                when WR_ADDR =>
                    if m_axil_awready = '1' then
                        phase <= WR_RESP;
                    end if;
                when WR_RESP =>
                    if m_axil_bvalid = '1' then
                        phase <= FINISH;
                    end if;
                when RD_ADDR =>
                    if m_axil_arready = '1' then
                        phase <= RD_DATA;
                    end if;
                when RD_DATA =>
                    if m_axil_rvalid = '1' then
                        rdata <= m_axil_rdata;
                        phase <= FINISH;
                    end if;
                when FINISH =>
                    if s_apb_penable = '1' then
                        phase <= IDLE;
                    end if;
            end case;
        end if;
    end if;
end process p_phase;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_phase	process	clk	52	—

2.8.1.6 apb_bridge — Clock Domain Analysis

Auto-extracted from `apb_bridge.vhd`.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

Clock	Clocked Processes
clk	p_phase

2.8.1.7 apb_bridge — Reset Domain Analysis

Auto-extracted from apb_bridge.vhd.

Note

All clocked processes use a single reset domain (rst, sync). No reset domain crossings detected.

Reset Domain

Process	Clock	Reset	Style
p_phase	clk	rst	Sync

2.8.2 axil_adapter

2.8.2.1 axil_adapter — Entity

Source file: src/axil_adapter.sv.

Flat AXI4-Lite to register file adapter.

Presents the register bus as flat AXI4-Lite subordinate ports (so a VHDL bridge can drive it) and wraps them in an axil_if instance for axil_regs.

Annotated Source

Listing 3: src/axil_adapter.sv

```
// Flat AXI4-Lite to register file adapter.
//
// Presents the register bus as flat AXI4-Lite subordinate ports (so a VHDL
// bridge can drive it) and wraps them in an axil_if instance for axil_regs.
module axil_adapter (
    // System clock.
    input  logic        clk,
    // Synchronous active-high reset.
    input  logic        rst,
    // AXI4-Lite subordinate port.
    input  logic [7:0]    s_axil_awaddr,
    input  logic          s_axil_awvalid,
    output logic          s_axil_awready,
    input  logic [31:0]   s_axil_wdata,
    input  logic [3:0]    s_axil_wstrb,
    input  logic          s_axil_wvalid,
    output logic          s_axil_wready,
```

(continues on next page)

(continued from previous page)

```
    output logic [1:0]    s_axil_bresp,
    output logic          s_axil_bvalid,
    input  logic          s_axil_bready,
    input  logic [7:0]    s_axil_araddr,
    input  logic          s_axil_arvalid,
    output logic          s_axil_arready,
    output logic [31:0]   s_axil_rdata,
    output logic [1:0]    s_axil_rresp,
    output logic          s_axil_rvalid,
    input  logic          s_axil_rready,
    // Starts a capture while high.
    output logic        start,
    // Enables the stream filters.
    output logic        filter_enable,
    // Holds the packet counter at zero while high.
    output logic        clear,
    // Clears the sampler's overflow flag while high.
    output logic        clear_overflow,
    // Smallest first-byte value the filters let through.
    output logic [7:0]  threshold,
    // Sampler controller state.
    input  logic [1:0]  state_code,
    // A sample was lost because the FIFO was full.
    input  logic        overflow,
    // Packets forwarded since the counter was last cleared.
    input  logic [15:0] count
);
    axil_if bus ();

    assign bus.awaddr = s_axil_awaddr;
    assign bus.awvalid = s_axil_awvalid;
    assign s_axil_awready = bus.awready;
    assign bus.wdata = s_axil_wdata;
    assign bus.wstrb = s_axil_wstrb;
    assign bus.wvalid = s_axil_wvalid;
    assign s_axil_wready = bus.wready;
    assign s_axil_bresp = bus.bresp;
    assign s_axil_bvalid = bus.bvalid;
    assign bus.bready = s_axil_bready;
    assign bus.araddr = s_axil_araddr;
    assign bus.arvalid = s_axil_arvalid;
    assign s_axil_arready = bus.arready;
    assign s_axil_rdata = bus.rdata;
    assign s_axil_rresp = bus.rresp;
    assign s_axil_rvalid = bus.rvalid;
    assign bus.rready = s_axil_rready;

    axil_regs u_regs (
        .clk(clk), .rst(rst), .s_axil(bus),
        .start(start), .filter_enable(filter_enable), .clear(clear), .clear_
↪overflow(clear_overflow),
        .threshold(threshold), .state_code(state_code), .overflow(overflow), .
```

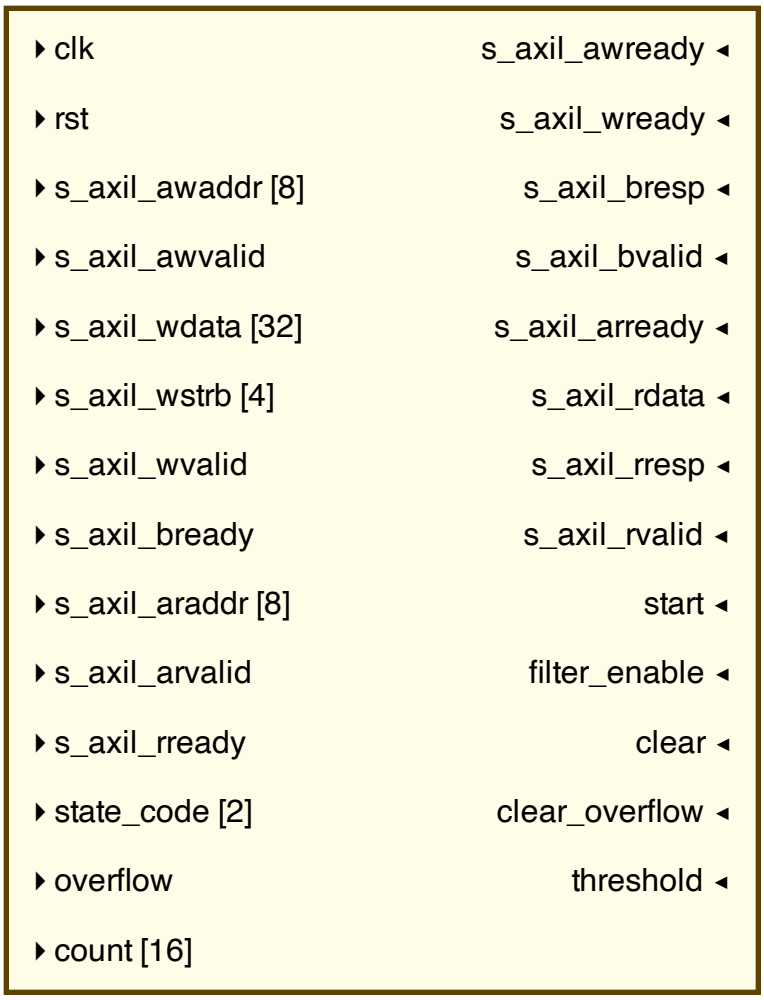
(continues on next page)

(continued from previous page)

```
↪count(count)
    );
endmodule
```

2.8.2.2 axil_adapter — Block Diagram

Auto-extracted from `axil_adapter.sv`.



Ports

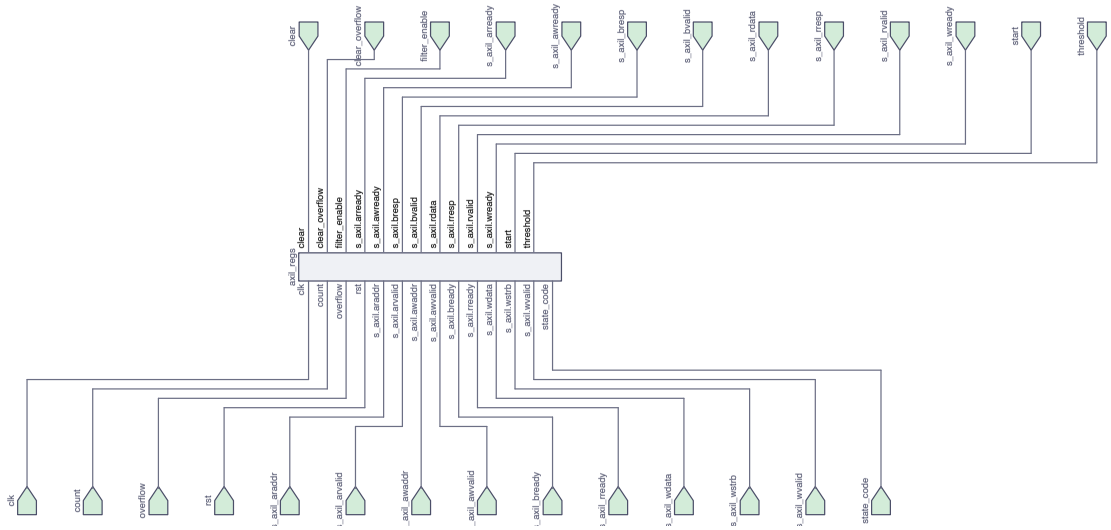
Interfaces

`axil_if` — instantiated here as bus.

Signal	Type	slave	master	Description
awaddr	logic[full_pkg::ADDR_W-1:0]	in	out	Write address channel.
awvalid	logic	in	out	—
awready	logic	out	in	—
wdata	logic[31:0]	in	out	Write data channel.
wstrb	logic[3:0]	in	out	—
wvalid	logic	in	out	—
wready	logic	out	in	—
bresp	logic[1:0]	out	in	Write response channel.
bvalid	logic	out	in	—
bready	logic	in	out	—
araddr	logic[full_pkg::ADDR_W-1:0]	in	out	Read address channel.
arvalid	logic	in	out	—
arready	logic	out	in	—
rdata	logic[31:0]	out	in	Read data channel.
rresp	logic[1:0]	out	in	—
rvalid	logic	out	in	—
rready	logic	in	out	—

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


clear
clear_overflow
filter_enable
s_axil_arready
s_axil_awready
s_axil_bresp
s_axil_bvalid
s_axil_rdata
s_axil_rresp
s_axil_rvalid
s_axil_wready
start
threshold
axil_regs
clk
count
overflow
rst
s_axil_araddr
s_axil_arvalid
s_axil_awaddr
s_axil_awvalid
s_axil_bready
s_axil_rready
s_axil_wdata
s_axil_wstrb
s_axil_wvalid
state_code
clk
count
overflow
rst
s_axil_araddr
s_axil_arvalid
s_axil_awaddr
s_axil_awvalid
s_axil_bready
s_axil_rready
s_axil_wdata
s_axil_wstrb
s_axil_wvalid
state_code

2.8.2.3 axil_adapter — State Machine

No FSM detected in `axil_adapter.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.8.2.4 axil_adapter — Timing Diagrams

All timing diagrams extracted from `axil_adapter.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.8.2.5 axil_adapter — Clock Domain Analysis

Auto-extracted from `axil_adapter.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.8.2.6 axil_adapter — Reset Domain Analysis

Auto-extracted from `axil_adapter.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.8.2.7 Submodules

axil_regs

axil_regs — Entity

Source file: `src/axil_regs.sv`.

AXI4-Lite register file for the sampler.

Five 32-bit registers: control (start, filter enable, clear), a read-only status word (controller state, overflow flag), the filter threshold, a scratch register and a read-only packet count. Write strobes are ignored: every write updates the whole register. See registers/regs_axil_regs.toml.

Annotated Source

Listing 4: src/axil_regs.sv

```
1 // AXI4-Lite register file for the sampler.
2 //
3 // Five 32-bit registers: control (start, filter enable, clear, clear overflow), a read-
  ↪only
4 // status word (controller state, overflow flag), the filter threshold, a
```

(continues on next page)

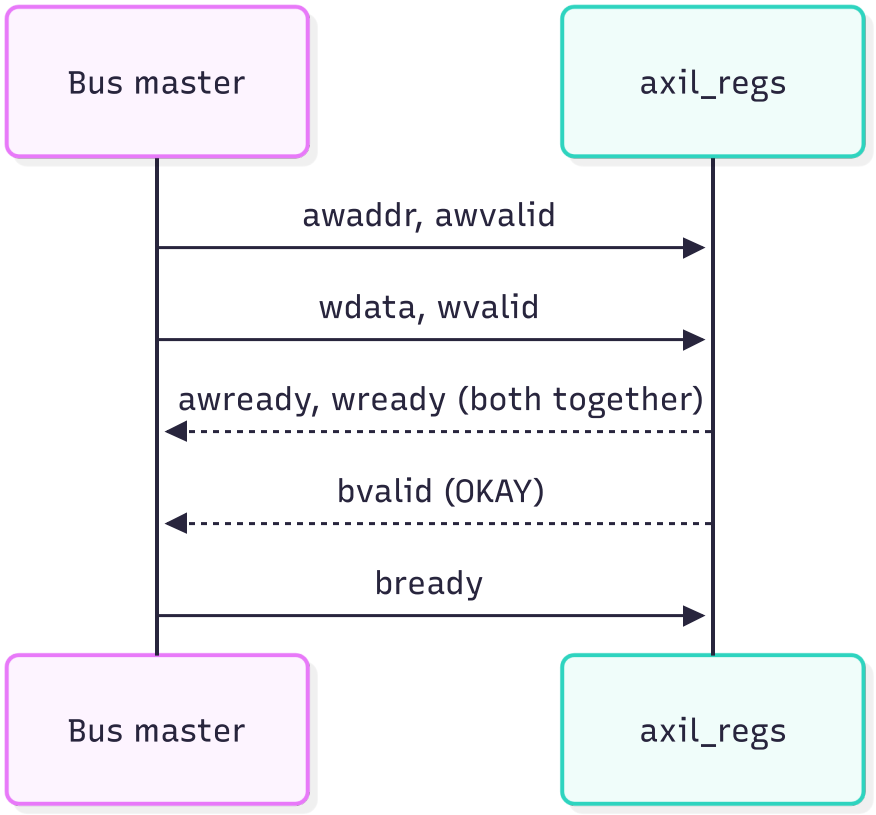


Fig. 2: AXI4-Lite register write

(continued from previous page)

```
// scratch register and a read-only packet count. Write strobes are ignored:
// every write updates the whole register. See registers/regs_axil_regs.toml.
//
// .. mermaid::
//    :caption: AXI4-Lite register write
//
//    sequenceDiagram
//      participant M as Bus master
//      participant R as axil_regs
//      M->>R: awaddr, awvalid
//      M->>R: wdata, wvalid
//      R-->>M: awready, wready (both together)
//      R-->>M: bvalid (OKAY)
//      M->>R: bready
module axil_regs (
    // System clock.
    input  logic        clk,
    // Synchronous active-high reset.
    input  logic        rst,
    // Register access from the bus master.
    axil_if.slave       s_axil,
    // Starts a capture while high.
    output logic        start,
    // Enables the stream filters.
    output logic        filter_enable,
    // Holds the packet counter at zero while high.
    output logic        clear,
    // Clears the sampler's overflow flag while high.
    output logic        clear_overflow,
    // Smallest first-byte value the filters let through.
    output logic [7:0]  threshold,
    // Sampler controller state.
    input  logic [1:0]  state_code,
    // A sample was lost because the FIFO was full.
    input  logic        overflow,
    // Packets forwarded since the counter was last cleared.
    input  logic [15:0] count
);
    logic [31:0] ctrl_r, threshold_r, scratch_r, rdata_r;
    logic        bvalid_r, rvalid_r;

    // Interface members are copied to plain signals first: Yosys cannot take a bit-
↪select
    // of an interface member inside a submodule.
    wire [7:0]  awaddr = s_axil.awaddr;
    wire [7:0]  araddr = s_axil.araddr;
    wire [31:0] wdata  = s_axil.wdata;

    // A write is taken when both address and data are present and no response is␣
↪pending.
    wire wr_fire = s_axil.awvalid && s_axil.wvalid && !bvalid_r;
    wire rd_fire = s_axil.arvalid && !rvalid_r;
```

(continues on next page)

(continued from previous page)

```
    // p_bus: Decodes writes and reads and drives the responses.
    always_ff @(posedge clk) begin : p_bus
        if (rst) begin
            ctrl_r      <= 32'd0;
            threshold_r <= 32'd0;
            scratch_r   <= 32'd0;
            rdata_r     <= 32'd0;
            bvalid_r    <= 1'b0;
            rvalid_r    <= 1'b0;
        end else begin
            if (wr_fire) begin
                bvalid_r <= 1'b1;
                case (awaddr[4:2])
                    3'd0:    ctrl_r      <= wdata;
                    3'd2:    threshold_r <= wdata;
                    3'd3:    scratch_r   <= wdata;
                    default: ;
                endcase
            end else if (s_axil.bready) begin
                bvalid_r <= 1'b0;
            end

            if (rd_fire) begin
                rvalid_r <= 1'b1;
                case (araddr[4:2])
                    3'd0:    rdata_r <= ctrl_r;
                    3'd1:    rdata_r <= {29'd0, overflow, state_code};
                    3'd2:    rdata_r <= threshold_r;
                    3'd3:    rdata_r <= scratch_r;
                    default: rdata_r <= {16'd0, count};
                endcase
            end else if (s_axil.rready) begin
                rvalid_r <= 1'b0;
            end
        end
    end

    assign s_axil.awready = wr_fire;
    assign s_axil.wready  = wr_fire;
    assign s_axil.bresp   = 2'b00;
    assign s_axil.bvalid  = bvalid_r;
    assign s_axil.arready = rd_fire;
    assign s_axil.rdata   = rdata_r;
    assign s_axil.rresp   = 2'b00;
    assign s_axil.rvalid  = rvalid_r;

    assign start         = ctrl_r[0];
    assign filter_enable = ctrl_r[1];
    assign clear         = ctrl_r[2];
    assign clear_overflow = ctrl_r[3];
    assign threshold     = threshold_r[7:0];
```

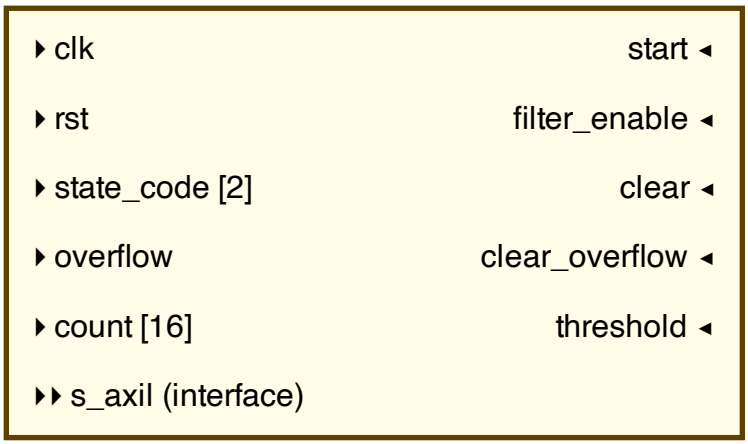
(continues on next page)

(continued from previous page)

```
107 endmodule
```

axil_regs — Block Diagram

Auto-extracted from `axil_regs.sv`.



Ports

Port	Direction	Type	Description
`clk`	`in`	`logic`	System clock.
`rst`	`in`	`logic`	Synchronous active-high reset.
`s_axil`	`slave`	`axil_if`	Register access from the bus master.
`start`	`out`	`logic`	Starts a capture while high.
`filter_enable`	`out`	`logic`	Enables the stream filters.
`clear`	`out`	`logic`	Holds the packet counter at zero while high.
`clear_overflow`	`out`	`logic`	Clears the sampler's overflow flag while high.
`threshold`	`out`	`logic[7:0]`	Smallest first-byte value the filters let through.
`state_code`	`in`	`logic[1:0]`	Sampler controller state.
`overflow`	`in`	`logic`	A sample was lost because the FIFO was full.
`count`	`in`	`logic[15:0]`	Packets forwarded since the counter was last cleared.

Interfaces

axil_if — used by s_axil.

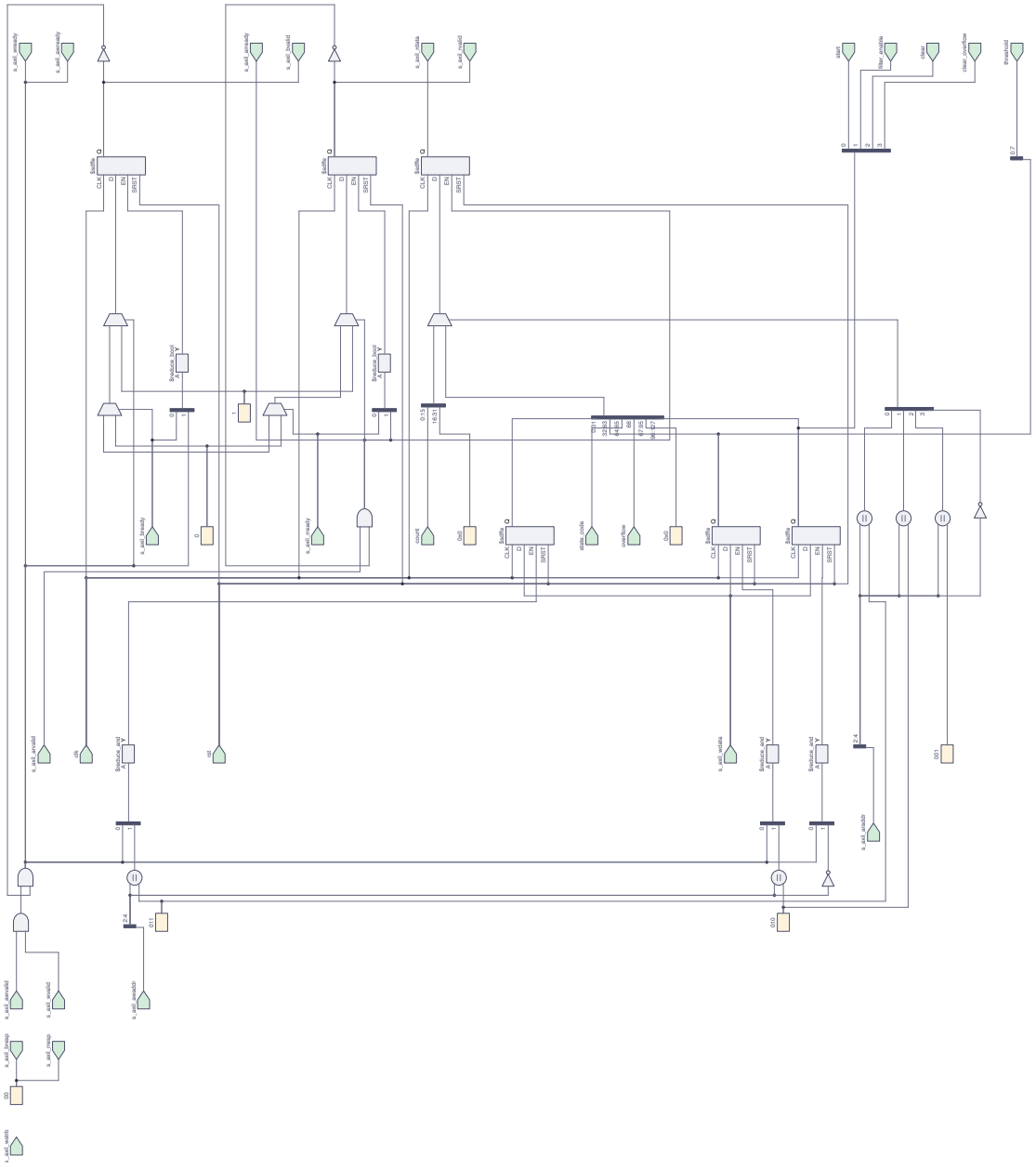
Signal	Type	slave	Description
awaddr	logic[full_pkg::ADDR_W-1:0]	in	Write address channel.
awvalid	logic	in	—
awready	logic	out	—
wdata	logic[31:0]	in	Write data channel.
wstrb	logic[3:0]	in	—
wvalid	logic	in	—
wready	logic	out	—
bresp	logic[1:0]	out	Write response channel.
bvalid	logic	out	—
bready	logic	in	—
araddr	logic[full_pkg::ADDR_W-1:0]	in	Read address channel.
arvalid	logic	in	—
arready	logic	out	—
rdata	logic[31:0]	out	Read data channel.
rresp	logic[1:0]	out	—
rvalid	logic	out	—
rready	logic	in	—

Signals

Signal	Type	Description
ctrl_r	logic[31:0]	—
threshold_r	logic[31:0]	—
scratch_r	logic[31:0]	—
rdata_r	logic[31:0]	—
bvalid_r	logic	—
rvalid_r	logic	—

RTL Detail

Gate-level netlist generated by yosys and rendered with netlistsvg.



axil_regs — State Machine

No FSM detected in `axil_regs.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

axil_regs — Timing Diagrams

All timing diagrams extracted from `axil_regs.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

Processes

Auto-extracted from `axil_regs.sv`.

p_bus

Source file: `axil_regs.sv`
Source line: 57
Block type: `always_ff`
Sensitivity list: `posedge clk`

Description

p_bus: Decodes writes and reads and drives the responses.

Signal Assignments

State	bvalid_r	rvalid_r
DEFAULT	0	0

Source

```
always_ff @(posedge clk) begin : p_bus
    if (rst) begin
        ctrl_r      <= 32'd0;
        threshold_r <= 32'd0;
        scratch_r   <= 32'd0;
        rdata_r     <= 32'd0;
        bvalid_r    <= 1'b0;
        rvalid_r    <= 1'b0;
    end else begin
        if (wr_fire) begin
            bvalid_r <= 1'b1;
            case (awaddr[4:2])
                3'd0:    ctrl_r      <= wdata;
```

(continues on next page)

(continued from previous page)

```
                3'd2:    threshold_r <= wdata;
                3'd3:    scratch_r   <= wdata;
                default: ;
            endcase
        end else if (s_axil.bready) begin
            bvalid_r <= 1'b0;
        end

        if (rd_fire) begin
            rvalid_r <= 1'b1;
            case (araddr[4:2])
                3'd0:    rdata_r <= ctrl_r;
                3'd1:    rdata_r <= {29'd0, overflow, state_code};
                3'd2:    rdata_r <= threshold_r;
                3'd3:    rdata_r <= scratch_r;
                default: rdata_r <= {16'd0, count};
            endcase
        end else if (s_axil.rready) begin
            rvalid_r <= 1'b0;
        end
    end
end
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_bus	always_ff	posedge clk	57	—

axil_regs — Clock Domain Analysis

Auto-extracted from `axil_regs.sv`.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

Clock	Clocked Processes
clk	p_bus

axil_regs — Reset Domain Analysis

Auto-extracted from `axil_regs.sv`.

Note

All clocked processes use a single reset domain (`rst`, sync). No reset domain crossings detected.

Reset Domain

Process	Clock	Reset	Style
p_bus	clk	rst	Sync

axil_regs — Register Map

Register	Offset	Access	Description
ctrl	0x00	r/w	Control register for the sampler.
status	0x04	r	Sampler status.
threshold	0x08	r/w	Filter threshold.
scratch	0x0C	r/w	General-purpose scratch register. Reads back what was written; no effect on the design.
count	0x10	r	Forwarded-packet count.

ctrl (0x00)

Field	Type	Default	Description
start	bit	0	Assert to start a capture. Hold until the status reports done, then clear.
filter_enabl	bit	0	Assert to let the stream filters forward packets. While low every packet is dropped.
clear	bit	0	Hold high to hold the forwarded-packet counter at zero.
clear_overfl	bit	0	Hold high to clear the overflow flag. While the flag is set a new capture ends at once, so clear it before starting.

status (0x04)

Field	Type	Default	Description
state	bit_vector(2)		Controller state: 0 idle, 1 armed, 2 running, 3 done.
overflow	bit		A sample was lost because the capture FIFO was full.

threshold (0x08)

Field	Type	Default	Description
value	bit_vector(8)	00000000	Smallest first-byte value that lets a packet through. Packets below it are dropped.

scratch (0x0C)

Field	Type	Default	Description
value	bit_vector(32)	00000000000000000(	Software scratch value.

count (0x10)

Field	Type	Default	Description
forwarded	bit_vector(16)		Packets forwarded since the counter was last cleared.

2.8.3 axis_pipeline

2.8.3.1 axis_pipeline — Entity

Source file: src/axis_pipeline.sv.

Filtered pass-through for an external byte stream.

Presents flat AXI4-Stream ports, wraps them in interface instances and runs them through a stream_filter.

Annotated Source

Listing 5: src/axis_pipeline.sv

```
1  // Filtered pass-through for an external byte stream.
2  //
3  // Presents flat AXI4-Stream ports, wraps them in interface instances and runs
4  // them through a stream_filter.
5  module axis_pipeline (
6      // System clock.
7      input  logic       clk,
8      // Synchronous active-high reset.
9      input  logic       rst,
10     // Filtering is on while high.
11     input  logic       enable,
12     // Holds the packet counter at zero while high.
13     input  logic       clear,
14     // Smallest first-byte value that lets a packet through.
15     input  logic [7:0] threshold,
16     // Incoming bytes.
17     input  logic [7:0] s_axis_tdata,
18     input  logic       s_axis_tvalid,
```

(continues on next page)

(continued from previous page)

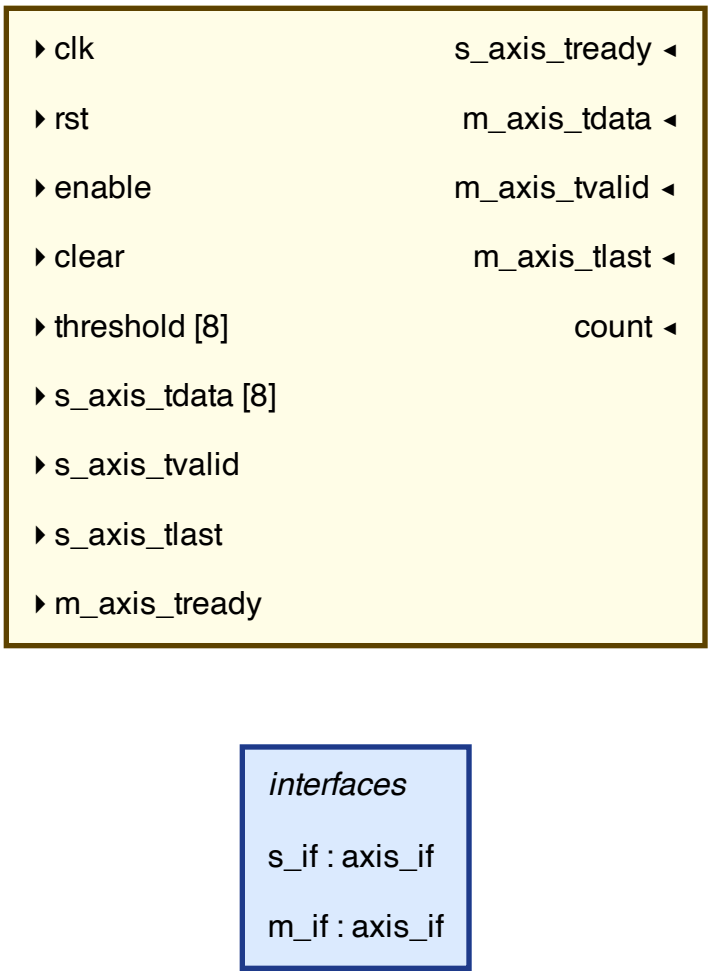
```
    output logic       s_axis_tready,
    input  logic       s_axis_tlast,
    // Forwarded bytes.
    output logic [7:0] m_axis_tdata,
    output logic       m_axis_tvalid,
    input  logic       m_axis_tready,
    output logic       m_axis_tlast,
    // Packets forwarded since the counter was last cleared.
    output logic [15:0] count
);
    axis_if s_if ();
    axis_if m_if ();

    assign s_if.tdata  = s_axis_tdata;
    assign s_if.tvalid = s_axis_tvalid;
    assign s_if.tlast  = s_axis_tlast;
    assign s_axis_tready = s_if.tready;
    assign m_axis_tdata  = m_if.tdata;
    assign m_axis_tvalid = m_if.tvalid;
    assign m_axis_tlast  = m_if.tlast;
    assign m_if.tready   = m_axis_tready;

    stream_filter u_filter (
        .clk(clk), .rst(rst), .enable(enable), .clear(clear), .threshold(threshold),
        .s_axis(s_if), .m_axis(m_if), .count(count)
    );
endmodule
```

2.8.3.2 axis_pipeline — Block Diagram

Auto-extracted from `axis_pipeline.sv`.



Ports

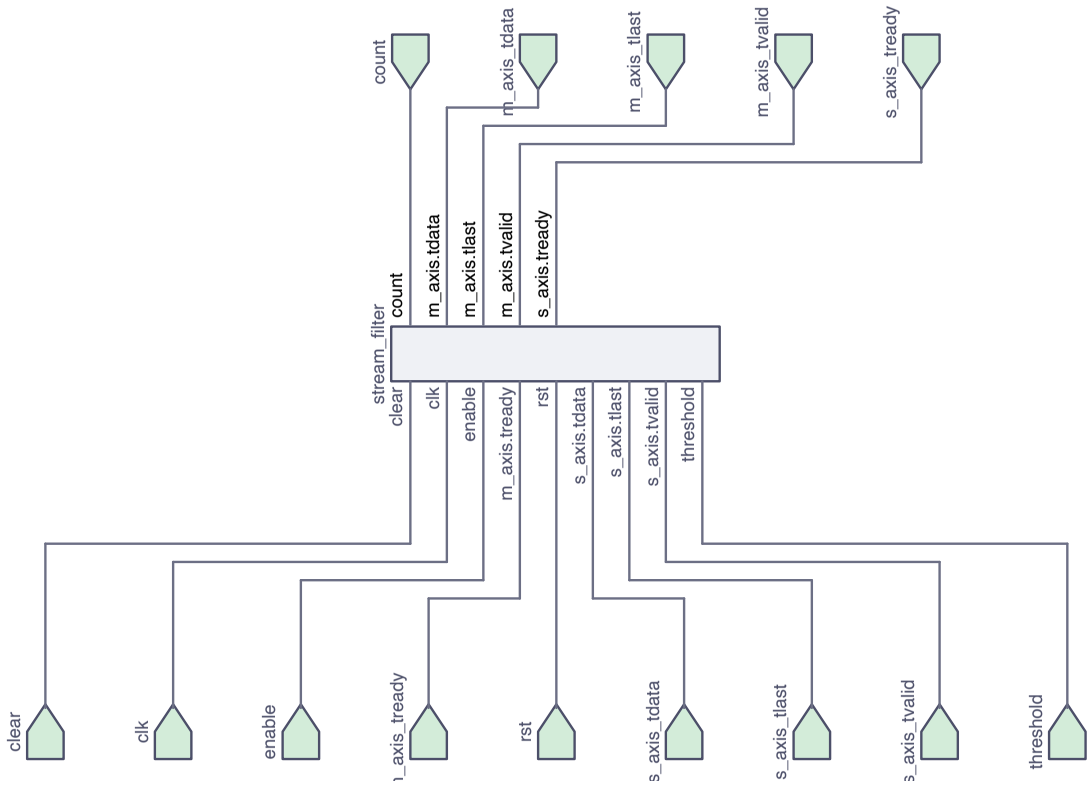
Interfaces

`axis_if` — instantiated here as `s_if`, `m_if`.

Signal	Type	master	slave	Description
`tdata`	`logic[7:0]`	out	in	Payload byte.
`tvalid`	`logic`	out	in	Master has a byte; slave takes it when tready is also high.
`tready`	`logic`	in	out	Slave can take a byte this cycle.
`tlast`	`logic`	out	in	Last byte of a packet.

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


count
m_axis_tdata
m_axis_tlast
m_axis_tvalid
s_axis_tready
stream_filter
count
m_axis.tdata
m_axis.tlast
m_axis.tvalid
s_axis.tready
clear
clk
enable
m_axis.tready
rst
s_axis.tdata
s_axis.tlast
s_axis.tvalid
threshold
clear
clk
enable
rst
threshold

2.8.3.3 axis_pipeline — State Machine

No FSM detected in `axis_pipeline.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.8.3.4 axis_pipeline — Timing Diagrams

All timing diagrams extracted from `axis_pipeline.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.8.3.5 axis_pipeline — Clock Domain Analysis

Auto-extracted from `axis_pipeline.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.8.3.6 axis_pipeline — Reset Domain Analysis

Auto-extracted from `axis_pipeline.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.8.3.7 Submodules

stream_filter

stream_filter — Entity

Source file: `src/stream_filter.sv`.

Packet filter for a byte stream.

Looks at the first byte of each packet: if it is at or above the threshold (and the filter is enabled) the whole packet is forwarded, otherwise the whole packet is dropped. Counts the packets it forwards.

Annotated Source

Listing 6: src/stream_filter.sv

```
1  // Packet filter for a byte stream.
2  //
3  // Looks at the first byte of each packet: if it is at or above the threshold
4  // (and the filter is enabled) the whole packet is forwarded, otherwise the
5  // whole packet is dropped. Counts the packets it forwards.
6  module stream_filter #(
```

(continues on next page)

(continued from previous page)

```
    // Width of the forwarded-packet counter in bits.
    parameter int COUNT_W = 16
) (
    // System clock.
    input  logic                clk,
    // Synchronous active-high reset.
    input  logic                rst,
    // Filtering is on while high; while low every packet is dropped.
    input  logic                enable,
    // Holds the counter at zero while high.
    input  logic                clear,
    // Smallest first-byte value that lets a packet through.
    input  logic [7:0]          threshold,
    // Incoming bytes.
    axis_if.slave               s_axis,
    // Forwarded bytes.
    axis_if.master              m_axis,
    // Packets forwarded since the counter was last cleared.
    output logic [COUNT_W-1:0] count
);
    typedef enum logic [1:0] {
        // Waiting for the first byte of a packet.
        FIRST,
        // Forwarding the rest of an accepted packet.
        FORWARD,
        // Dropping the rest of a rejected packet.
        DISCARD
    } state_t;

    state_t state, next_state;

    wire [7:0] tdata  = s_axis.tdata;
    wire       tvalid = s_axis.tvalid;
    wire       tlast  = s_axis.tlast;
    wire       first_ok   = enable && (tdata >= threshold);
    wire       in_first   = (state == FIRST);
    wire       forwarding = (state == FORWARD) || (in_first && first_ok);
    wire       beat       = tvalid && (m_axis.tready || !forwarding);
    wire       start_fwd  = in_first && beat && first_ok && !tlast;
    wire       start_drop = in_first && beat && !first_ok && !tlast;
    wire       last_beat  = beat && tlast;

    // p_state: Clocked state register and forwarded-packet counter.
    //
    // .. wavedrom::
    //
    //    { "signal": [
    //      { "name": "clk",    "wave": "P......." },
    //      { "name": "tvalid", "wave": "01.....0" },
    //      { "name": "tlast",  "wave": "0....10." },
    //      { "name": "state",  "wave": "=.=...=.", "data": ["FIRST","FORWARD","FIRST"] }
    //    ]}
```

(continues on next page)

(continued from previous page)

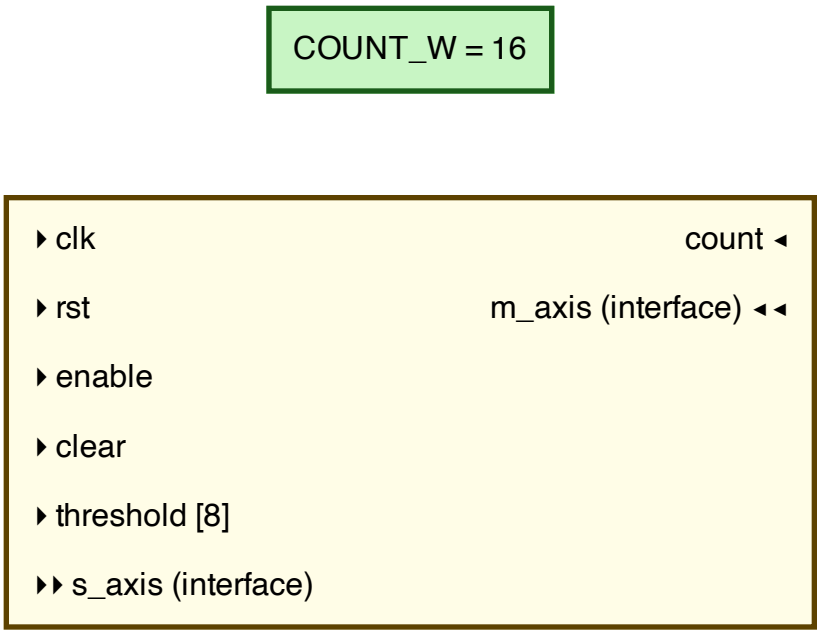
```
    always_ff @(posedge clk) begin : p_state
        if (rst) begin
            state <= FIRST;
            count <= '0;
        end else begin
            state <= next_state;
            if (clear) begin
                count <= '0;
            end else if (forwarding && last_beat) begin
                count <= count + 1'b1;
            end
        end
    end

    // p_next_state: Packet-level next-state logic.
    //
    // The first byte of a packet decides whether the rest is forwarded or
    // dropped; the last byte returns to FIRST.
    always_comb begin : p_next_state
        next_state = state;
        case (state)
            FIRST:   if (start_fwd) next_state = FORWARD;
                     else if (start_drop) next_state = DISCARD;
            FORWARD: if (last_beat) next_state = FIRST;
            DISCARD: if (last_beat) next_state = FIRST;
            default: next_state = FIRST;
        endcase
    end

    assign m_axis.tdata  = tdata;
    assign m_axis.tvalid = tvalid && forwarding;
    assign m_axis.tlast  = tlast;
    assign s_axis.tready = forwarding ? m_axis.tready : 1'b1;
endmodule
```

stream_filter — Block Diagram

Auto-extracted from `stream_filter.sv`.



Ports

Port	Direction	Type	Description
`clk`	`in`	`logic`	System clock.
`rst`	`in`	`logic`	Synchronous active-high reset.
`enable`	`in`	`logic`	Filtering is on while high; while low every packet is dropped.
`clear`	`in`	`logic`	Holds the counter at zero while high.
`threshold`	`in`	`logic[7:0]`	Smallest first-byte value that lets a packet through.
`s_axis`	`slave`	`axis_if`	Incoming bytes.
`m_axis`	`master`	`axis_if`	Forwarded bytes.
`count`	`out`	`logic[COUNT_W-1:0]`	Packets forwarded since the counter was last cleared.

Interfaces

`axis_if` — used by `s_axis`, `m_axis`.

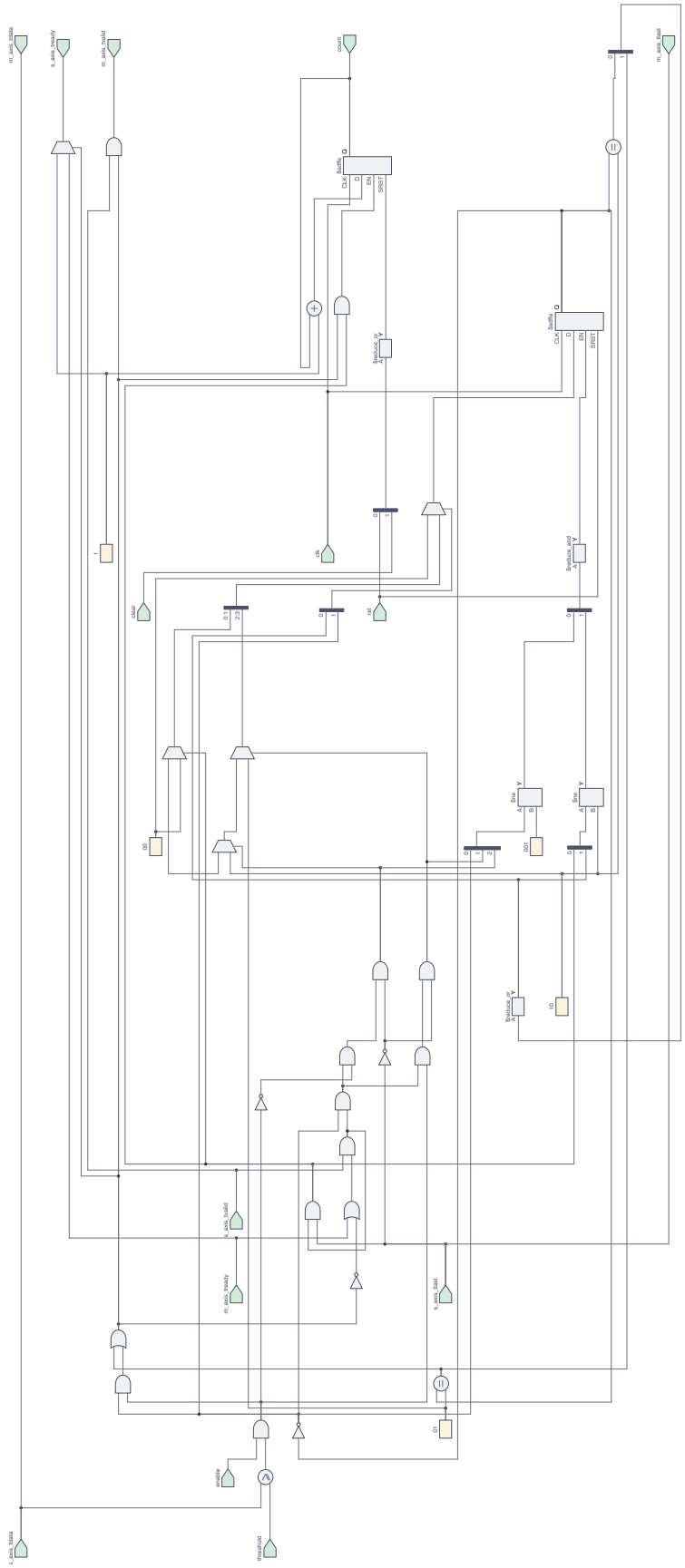
Signal	Type	slave	master	Description
`tdata`	`logic[7:0]`	in	out	Payload byte.
`tvalid`	`logic`	in	out	Master has a byte; slave takes it when tready is also high.
`tready`	`logic`	out	in	Slave can take a byte this cycle.
`tlast`	`logic`	in	out	Last byte of a packet.

Parameters

Name	Type	Default	Description
`COUNT_W`	`int`	`16`	Width of the forwarded-packet counter in bits.

RTL Detail

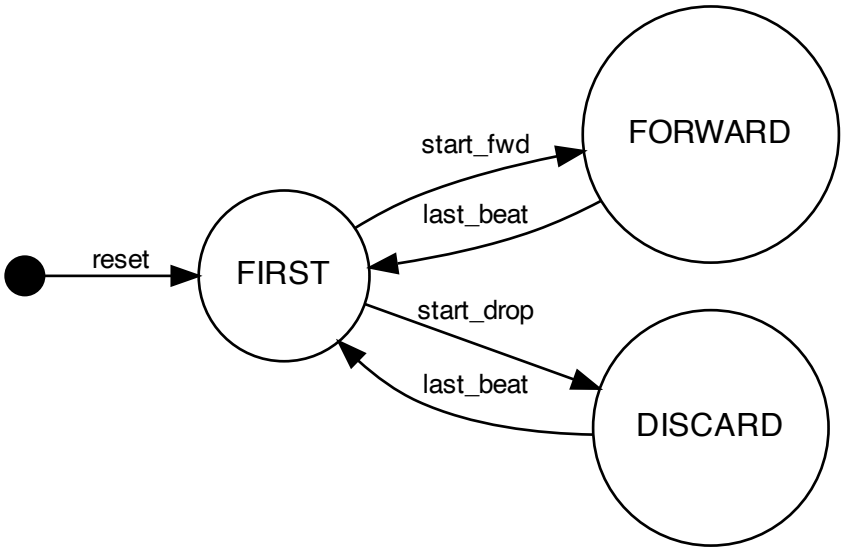
Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


m_axis_tdata
s_axis_tready
m_axis_tvalid
count
$auto$ff.cc
CLK
D
EN
SRST
Q
$reduce_or
A
Y
clk
rst
clear
$mux
A
B
00
001
10
01
s_axis_tvalid
m_axis_tready
s_axis_tlast
enable
threshold

stream_filter State Machine

Auto-extracted from `stream_filter.sv`.

State Transition Diagram



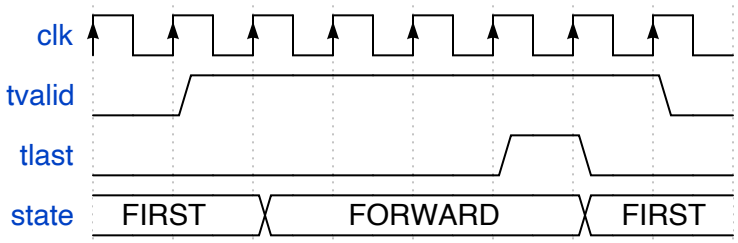
Transitions

From	To	Condition
FIRST	FORWARD	start_fwd
FIRST	DISCARD	start_drop
FORWARD	FIRST	last_beat
DISCARD	FIRST	last_beat

stream_filter — Timing Diagrams

All timing diagrams extracted from `stream_filter.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

P State



Processes

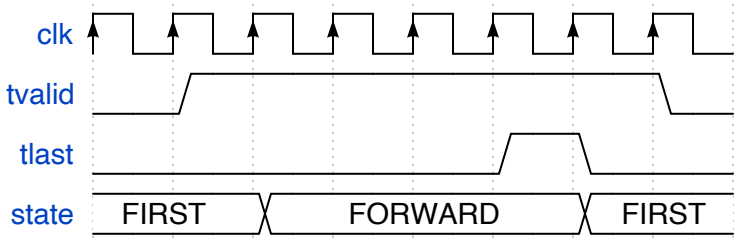
Auto-extracted from `stream_filter.sv`.

p_state

Source file: `stream_filter.sv`
Source line: 59
Block type: `always_ff`
Sensitivity list: `posedge clk`

Description

p_state: Clocked state register and forwarded-packet counter.



Source

```
always_ff @(posedge clk) begin : p_state
    if (rst) begin
        state <= FIRST;
        count <= '0;
    end else begin
        state <= next_state;
        if (clear) begin
            count <= '0;
        end else if (forwarding && last_beat) begin
            count <= count + 1'b1;
        end
    end
end
```

p_next_state

Source file: stream_filter.sv
Source line: 77
Block type: always_comb
Sensitivity list: *

Description

p_next_state: Packet-level next-state logic. The first byte of a packet decides whether the rest is forwarded or dropped; the last byte returns to FIRST.

Source

```
always_comb begin : p_next_state
    next_state = state;
    case (state)
        FIRST:   if (start_fwd) next_state = FORWARD;
                 else if (start_drop) next_state = DISCARD;
        FORWARD: if (last_beat) next_state = FIRST;
        DISCARD: if (last_beat) next_state = FIRST;
        default: next_state = FIRST;
    endcase
end
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_state	always_ff	posedge clk	59	✓
p_next_state	always_comb	*	77	—

stream_filter — Clock Domain Analysis

Auto-extracted from stream_filter.sv.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (clk).

Clock Domain

Clock	Clocked Processes
clk	p_state

stream_filter — Reset Domain Analysis

Auto-extracted from `stream_filter.sv`.

Note

All clocked processes use a single reset domain (`rst`, sync). No reset domain crossings detected.

Reset Domain

Process	Clock	Reset	Style
`p_state`	`clk`	`rst`	Sync

Note

Modules marked **shared** are instantiated by multiple parents. Their documentation appears once; all parents link to the same page.

2.8.4 reset_sync

2.8.4.1 reset_sync — Entity

Source file: `src/reset_sync.vhd`.

ENTITY **`reset_sync`** IS

Reset synchroniser.

Takes an asynchronous active-low reset and delivers it to one clock domain: assertion is immediate, release is held until STAGES rising clock edges have passed, so every flop in the domain leaves reset on the same edge.

`Generics`

Generic	Type	Default	Description
STAGES	positive	2	Number of flip-flop stages in the release chain (2 or more).

`Ports`

Port	Type	Mode	Description
clk	std_logic	in	Clock of the domain the reset is delivered to.
arst_n	std_logic	in	Asynchronous active-low reset input.
srst_n	std_logic	out	Active-low reset, asserted asynchronously and released synchronously.

Annotated Source

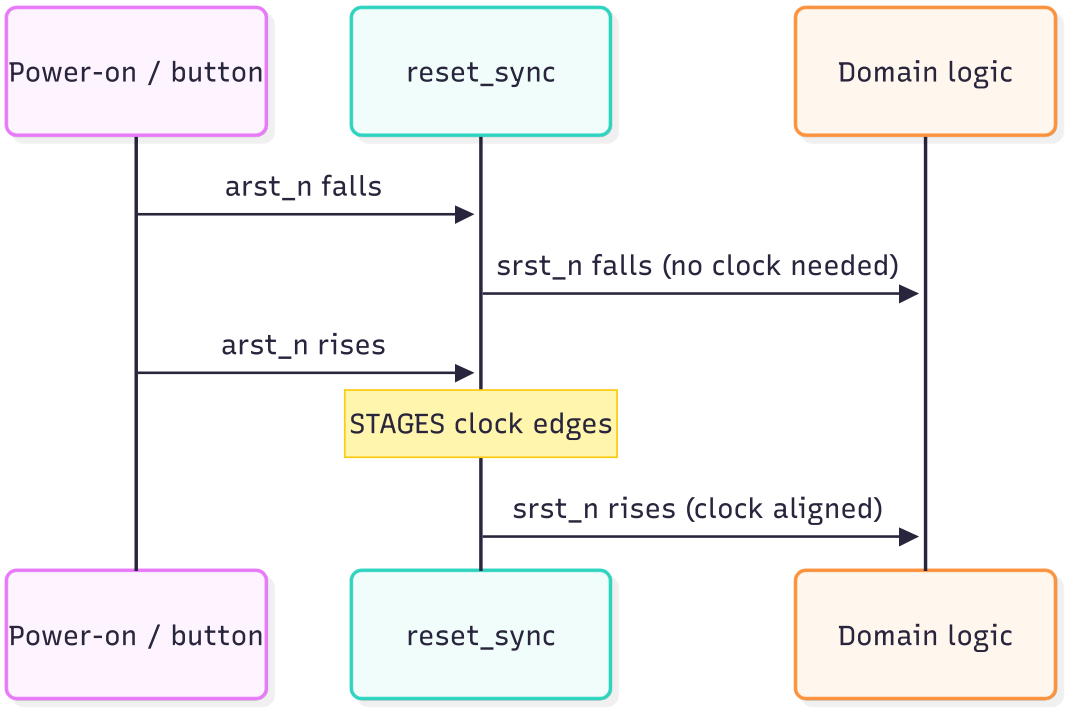


Fig. 3: Reset release

Listing 7: src/reset_sync.vhd

```vhdl
library ieee;
use ieee.std_logic_1164.all;

-- Reset synchroniser.
--
-- Takes an asynchronous active-low reset and delivers it to one clock domain:
-- assertion is immediate, release is held until STAGES rising clock edges have
-- passed, so every flop in the domain leaves reset on the same edge.
--
-- .. mermaid::
--    :caption: Reset release
--
--    sequenceDiagram
--      participant P as Power-on / button
--      participant S as reset_sync
--      participant D as Domain logic
--      P->>S: arst_n falls
--      S->>D: srst_n falls (no clock needed)
--      P->>S: arst_n rises
--      Note over S: STAGES clock edges
--      S->>D: srst_n rises (clock aligned)
entity reset_sync is
    generic (
        -- Number of flip-flop stages in the release chain (2 or more).
        STAGES : positive := 2
    );
    port (
        -- Clock of the domain the reset is delivered to.
        clk    : in  std_logic;
        -- Asynchronous active-low reset input.
        arst_n : in  std_logic;
        -- Active-low reset, asserted asynchronously and released synchronously.
        srst_n : out std_logic
    );
end entity reset_sync;

architecture rtl of reset_sync is
    -- Release chain, all cleared while arst_n is low.
    signal chain : std_logic_vector(STAGES - 1 downto 0) := (others => '0');
begin

    -- p_chain: Shifts a one through the chain once the reset is released.
    p_chain : process(clk, arst_n) is
    begin
        if arst_n = '0' then
            chain <= (others => '0');
        elsif rising_edge(clk) then
            chain <= chain(STAGES - 2 downto 0) & '1';
        end if;
    end process p_chain;

```

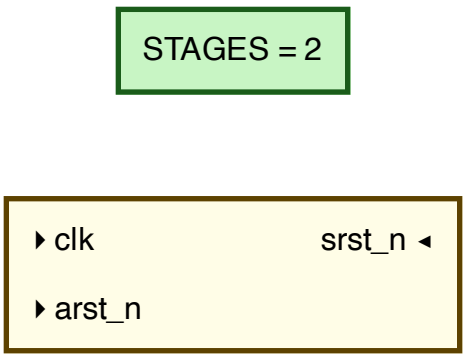
(continues on next page)

(continued from previous page)

```
52      srst_n <= chain(STAGES - 1);
53
54  end architecture rtl;
```

2.8.4.2 reset_sync — Block Diagram

Auto-extracted from `reset_sync.vhd`.



Ports

Port	Direction	Type	Description
`clk`	`in`	`std_logic`	Clock of the domain the reset is delivered to.
`arst_n`	`in`	`std_logic`	Asynchronous active-low reset input.
`srst_n`	`out`	`std_logic`	Active-low reset, asserted asynchronously and released synchronously.

Generics

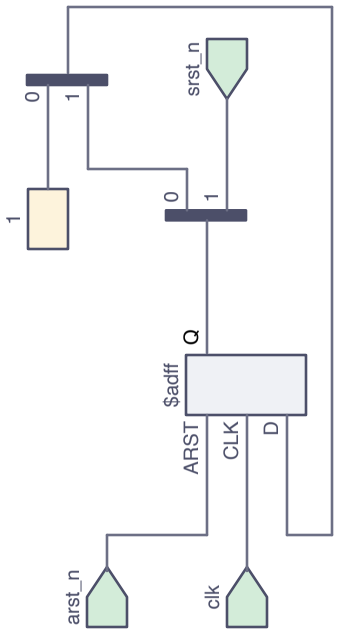
Name	Type	Default	Description
`STAGES`	`positive`	`2`	Number of flip-flop stages in the release chain (2 or more).

Signals

Signal	Type	Description
`chain`	`std_logic_vector(STAGES - 1 downto 0)`	Release chain, all cleared while arst_n is low.

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


srst_n
0
1
1
0
1
Q
$adff
ARST
CLK
D
arst_n
clk

2.8.4.3 reset_sync — State Machine

No FSM detected in `reset_sync.vhd`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.8.4.4 reset_sync — Timing Diagrams

All timing diagrams extracted from `reset_sync.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.8.4.5 Processes

Auto-extracted from `reset_sync.vhd`.

p_chain

Source file: `reset_sync.vhd`
Source line: 43
Block type: `process`
Sensitivity list: `clk`, `arst_n`

Description

p_chain: Shifts a one through the chain once the reset is released.

Source

```
p_chain : process(clk, arst_n) is
begin
    if arst_n = '0' then
        chain <= (others => '0');
    elsif rising_edge(clk) then
        chain <= chain(STAGES - 2 downto 0) & '1';
    end if;
end process p_chain;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_chain	process	clk, arst_n	43	—

2.8.4.6 reset_sync — Clock Domain Analysis

Auto-extracted from `reset_sync.vhd`.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

Clock	Clocked Processes
`clk`	`p_chain`

2.8.4.7 reset_sync — Reset Domain Analysis

Auto-extracted from `reset_sync.vhd`.

Note

All clocked processes use a single reset domain (`arst_n`, async). No reset domain crossings detected.

Reset Domain

Process	Clock	Reset	Style
`p_chain`	`clk`	`arst_n`	Async

2.8.5 sample_capture

2.8.5.1 sample_capture — Entity

Source file: `src/sample_capture.sv`.

Sample capture across a clock boundary.

Registers samples from a free-running converter clock (adc_clk) and hands them to the system clock (clk) through an asynchronous FIFO. The enable request goes the other way through a two-flop synchroniser, so neither domain samples a signal that changes under its own clock edge.

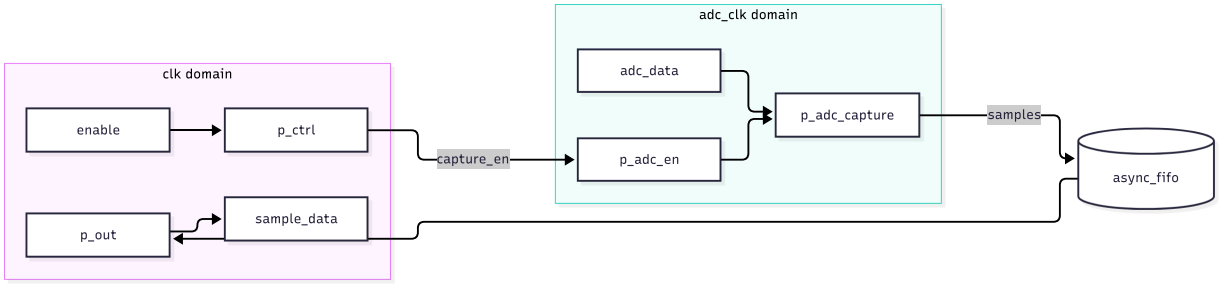


Fig. 4: Capture data path

Annotated Source

Listing 8: src/sample_capture.sv

```
// Sample capture across a clock boundary.
//
// Registers samples from a free-running converter clock (adc_clk) and hands
// them to the system clock (clk) through an asynchronous FIFO. The enable
// and clear requests go the other way through two-flop synchronisers, so
// neither domain samples a signal that changes under its own clock edge.
//
// .. mermaid::
//    :caption: Capture data path
//
//    flowchart LR
//      subgraph sys[clk domain]
//        E[enable] --> C[p_ctrl]
//        O[p_out] --> S[sample_data]
//      end
//      subgraph adc[adc_clk domain]
//        N[p_adc_en] --> R[p_adc_capture]
//        D[adc_data] --> R
//      end
//      C -- capture_en, clear_req --> N
//      R -- samples --> F[(async_fifo)]
//      F --> O
module sample_capture #(
    // Keep overflow set until reset (1), or show it for one cycle per lost
    // sample (0).
    parameter bit STICKY_OVERFLOW = 1'b1
) (
    // System clock.
    input  logic                       clk,
    // System-domain asynchronous active-low reset, released synchronously
    // (from a reset_sync on clk).
    input  logic                       rst_n,
    // Capture while high.
    input  logic                       enable,
    // Clears the overflow flag while high (system clock domain).
    input  logic                       clear_overflow,
    // Converter clock; unrelated to clk.
    input  logic                       adc_clk,
    // Converter-domain synchronous active-high reset, released on adc_clk.
    input  logic                       adc_rst,
    // Sample from the converter.
    input  logic [full_pkg::SAMPLE_W-1:0] adc_data,
    // Oldest unread sample, valid while sample_valid is high.
    output logic [full_pkg::SAMPLE_W-1:0] sample_data,
    // A sample is waiting on sample_data.
    output logic                       sample_valid,
    // Consumer takes the sample this cycle.
    input  logic                       sample_ready,
    // Set when a sample arrived while the FIFO was full (held until
```

(continues on next page)

(continued from previous page)

```
    // clear_overflow when STICKY_OVERFLOW). Changes under adc_clk, so the reader
    // must treat it as asynchronous.
    output logic                        overflow
);
    // Requests registered on clk: capture, and clear the overflow flag.
    logic                       capture_en, clear_req;
    // Two-flop synchronisers for capture_en and clear_req on adc_clk.
    logic                       en_meta, en_sync, clr_meta, clr_sync;
    // Registered sample and its write strobe on adc_clk.
    logic [full_pkg::SAMPLE_W-1:0] wr_data;
    logic                       wr_en;
    // FIFO handshake.
    logic                       fifo_full, fifo_empty;
    logic [full_pkg::SAMPLE_W-1:0] fifo_rd_data;
    wire                        rd_en     = !fifo_empty && (!sample_valid || sample_
↪ready);
    wire                        adc_rst_n = !adc_rst;

    // p_ctrl: Registers the enable and clear requests on the system clock.
    always_ff @(posedge clk or negedge rst_n) begin : p_ctrl
        if (!rst_n) begin
            capture_en <= 1'b0;
            clear_req  <= 1'b0;
        end else begin
            capture_en <= enable;
            clear_req  <= clear_overflow;
        end
    end

    // p_out: Presents the oldest sample to the consumer.
    always_ff @(posedge clk or negedge rst_n) begin : p_out
        if (!rst_n) begin
            sample_valid <= 1'b0;
            sample_data  <= '0;
        end else if (rd_en) begin
            sample_valid <= 1'b1;
            sample_data  <= fifo_rd_data;
        end else if (sample_ready) begin
            sample_valid <= 1'b0;
        end
    end

    // p_adc_en: Brings capture_en and clear_req into the converter domain.
    always_ff @(posedge adc_clk) begin : p_adc_en
        if (adc_rst) begin
            en_meta  <= 1'b0;
            en_sync  <= 1'b0;
            clr_meta <= 1'b0;
            clr_sync <= 1'b0;
        end else begin
            en_meta  <= capture_en;
            en_sync  <= en_meta;
```

(continues on next page)

(continued from previous page)

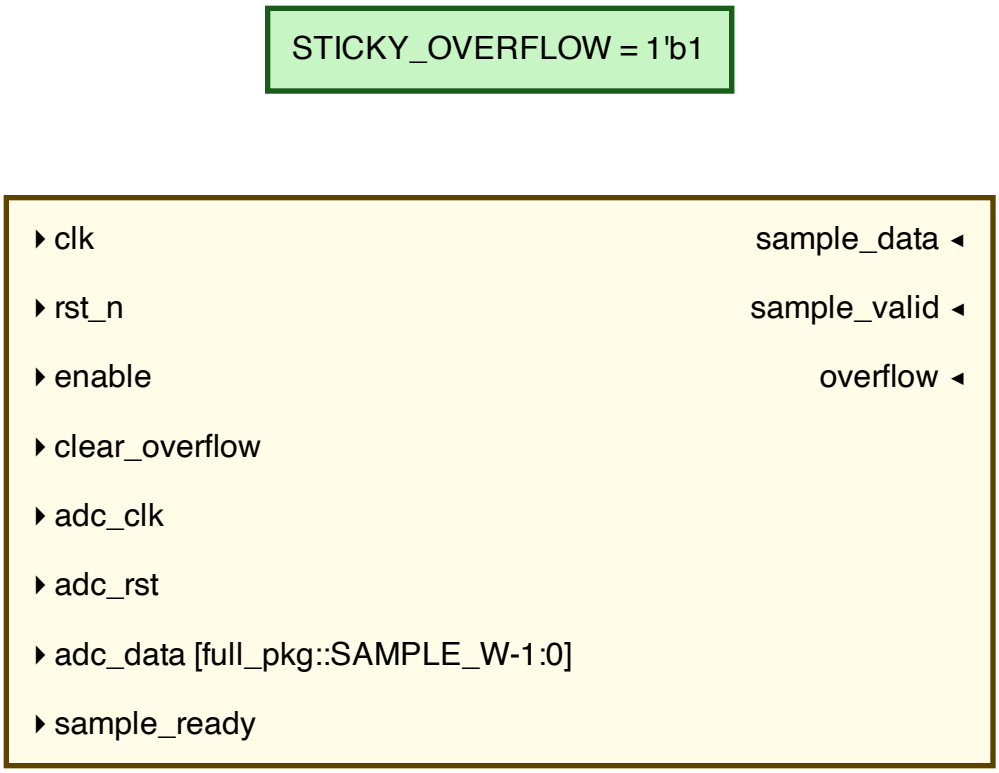
```
            clr_meta <= clear_req;
            clr_sync <= clr_meta;
        end
    end

    // p_adc_capture: Registers each sample and flags FIFO overflow.
    always_ff @(posedge adc_clk) begin : p_adc_capture
        if (adc_rst) begin
            wr_en    <= 1'b0;
            wr_data  <= '0;
            overflow <= 1'b0;
        end else begin
            wr_en   <= en_sync;
            wr_data <= adc_data;
            if (wr_en && fifo_full) begin
                overflow <= 1'b1;
            end else if (clr_sync || !STICKY_OVERFLOW) begin
                overflow <= 1'b0;
            end
        end
    end

    // Sample FIFO between the two clocks (VHDL; its generics are left at their
    // defaults, which match full_pkg::SAMPLE_W).
    async_fifo u_fifo (
        .wr_clk   (adc_clk),
        .wr_rst_n (adc_rst_n),
        .wr_en    (wr_en),
        .wr_data  (wr_data),
        .wr_full  (fifo_full),
        .rd_clk   (clk),
        .rd_rst_n (rst_n),
        .rd_en    (rd_en),
        .rd_data  (fifo_rd_data),
        .rd_empty (fifo_empty)
    );
endmodule
```

2.8.5.2 sample_capture — Block Diagram

Auto-extracted from `sample_capture.sv`.



Ports

Parameters

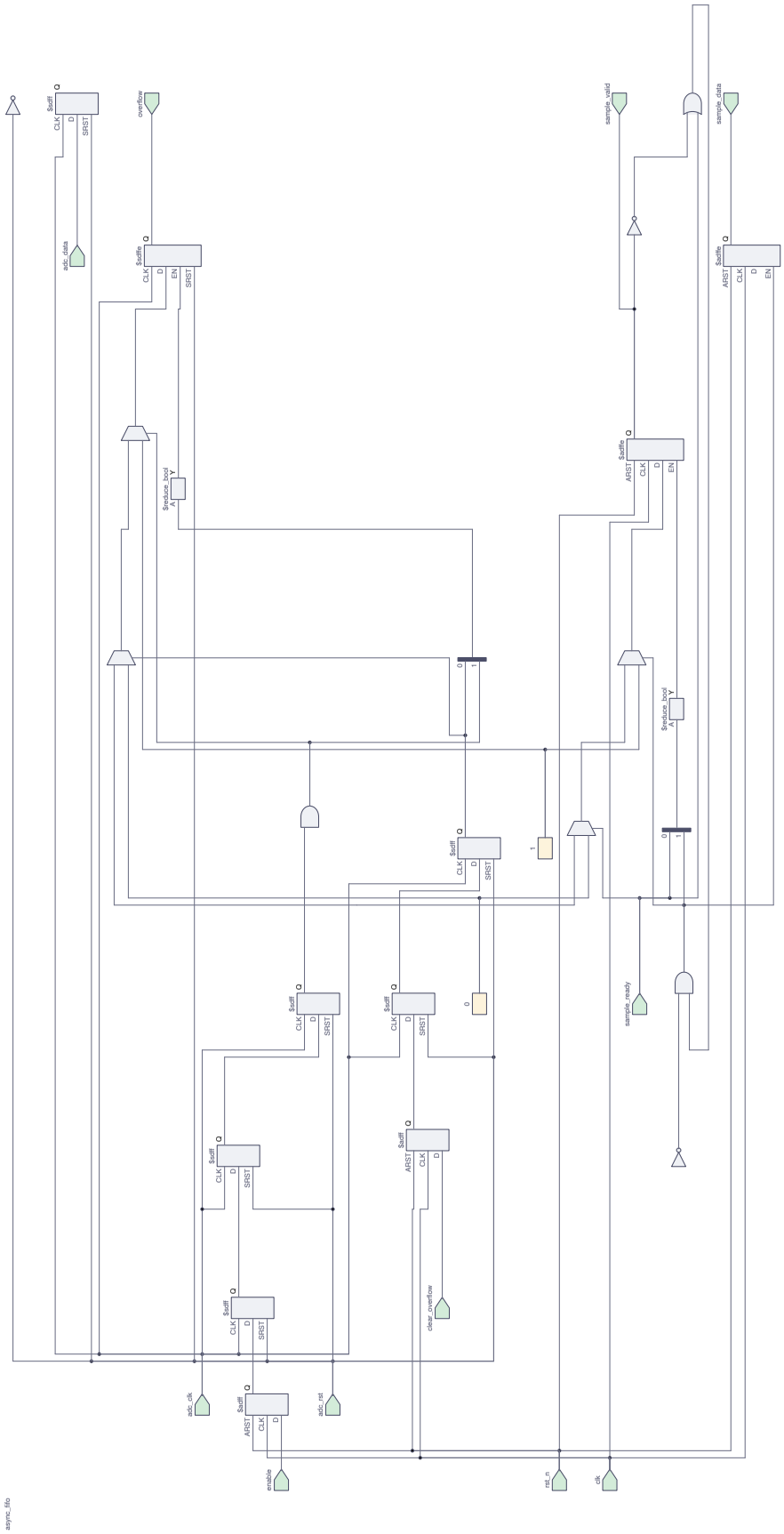
Name	Type	Default	Description
STICKY_OVERFLOW	bit	1'b1	sample (0).

Signals

Signal	Type	Description
capture_en	logic	Requests registered on clk: capture, and clear the overflow flag.
clear_req	logic	Requests registered on clk: capture, and clear the overflow flag.
en_meta	logic	Two-flop synchronisers for capture_en and clear_req on adc_clk.
en_sync	logic	Two-flop synchronisers for capture_en and clear_req on adc_clk.
clr_meta	logic	Two-flop synchronisers for capture_en and clear_req on adc_clk.
clr_sync	logic	Two-flop synchronisers for capture_en and clear_req on adc_clk.
wr_data	logic[full_pkg::SAMPLE_W-1:0	Registered sample and its write strobe on adc_clk.
wr_en	logic	—
fifo_full	logic	FIFO handshake.
fifo_empty	logic	FIFO handshake.
fifo_rd_data	logic[full_pkg::SAMPLE_W-1:0	—

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


$sdff
CLK
D
SRST
adc_data
overflow
$sdffe
CLK
D
EN
SRST
$reduce_bool
A
Y
sample_valid
sample_data
ARST
$sdffce
$adff
clear_overflow
sample_ready
adc_clk
adc_rst
enable
rst_n
clk
async_fifo

2.8.5.3 sample_capture — State Machine

No FSM detected in `sample_capture.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.8.5.4 sample_capture — Timing Diagrams

All timing diagrams extracted from `sample_capture.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.8.5.5 Processes

Auto-extracted from `sample_capture.sv`.

p_ctrl

Source file: `sample_capture.sv`
Source line: 68
Block type: `always_ff`
Sensitivity list: `posedge clk`, `negedge rst_n`

Description

p_ctrl: Registers the enable and clear requests on the system clock.

Source

```
always_ff @(posedge clk or negedge rst_n) begin : p_ctrl
    if (!rst_n) begin
        capture_en <= 1'b0;
        clear_req  <= 1'b0;
    end else begin
        capture_en <= enable;
        clear_req  <= clear_overflow;
    end
end
```

p_out

Source file: `sample_capture.sv`
Source line: 79
Block type: `always_ff`
Sensitivity list: `posedge clk`, `negedge rst_n`

Description

p_out: Presents the oldest sample to the consumer.

Source

```
always_ff @(posedge clk or negedge rst_n) begin : p_out
    if (!rst_n) begin
        sample_valid <= 1'b0;
        sample_data  <= '0;
    end else if (rd_en) begin
        sample_valid <= 1'b1;
        sample_data  <= fifo_rd_data;
    end else if (sample_ready) begin
        sample_valid <= 1'b0;
    end
end
```

p_adc_en

Source file: sample_capture.sv
Source line: 92
Block type: always_ff
Sensitivity list: posedge adc_clk

Description

p_adc_en: Brings capture_en and clear_req into the converter domain.

Source

```
always_ff @(posedge adc_clk) begin : p_adc_en
    if (adc_rst) begin
        en_meta  <= 1'b0;
        en_sync  <= 1'b0;
        clr_meta <= 1'b0;
        clr_sync <= 1'b0;
    end else begin
        en_meta  <= capture_en;
        en_sync  <= en_meta;
        clr_meta <= clear_req;
        clr_sync <= clr_meta;
    end
end
```

p_adc_capture

Source file: sample_capture.sv
Source line: 107
Block type: always_ff
Sensitivity list: posedge adc_clk

Description

p_adc_capture: Registers each sample and flags FIFO overflow.

Source

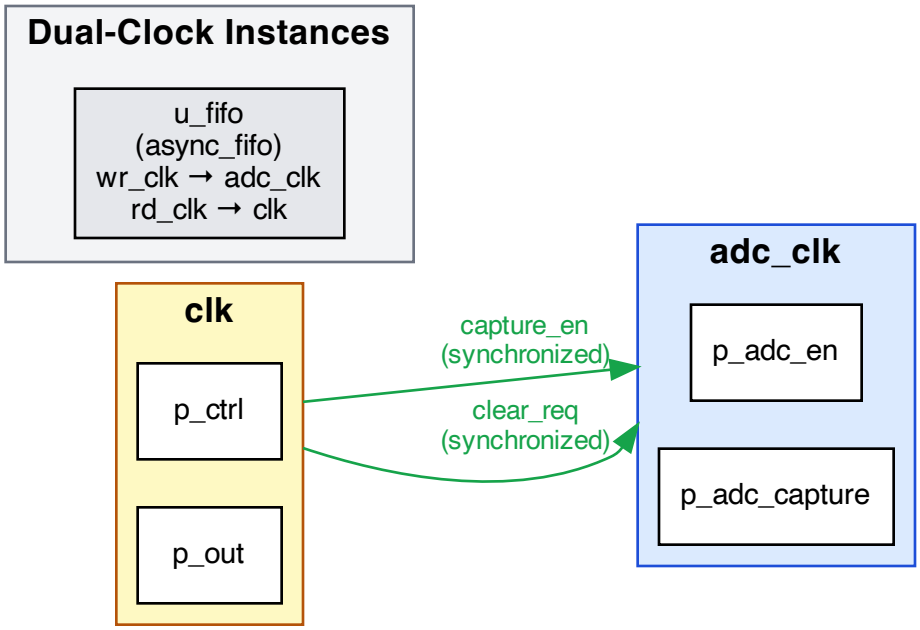
```
always_ff @(posedge adc_clk) begin : p_adc_capture
    if (adc_rst) begin
        wr_en    <= 1'b0;
        wr_data  <= '0;
        overflow <= 1'b0;
    end else begin
        wr_en   <= en_sync;
        wr_data <= adc_data;
        if (wr_en && fifo_full) begin
            overflow <= 1'b1;
        end else if (clr_sync || !STICKY_OVERFLOW) begin
            overflow <= 1'b0;
        end
    end
end
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_ctrl	always_ff	posedge clk, negedge rst_n	68	—
p_out	always_ff	posedge clk, negedge rst_n	79	—
p_adc_en	always_ff	posedge adc_clk	92	—
p_adc_capture	always_ff	posedge adc_clk	107	—

2.8.5.6 sample_capture — Clock Domain Analysis

Auto-extracted from `sample_capture.sv`.



Clock Domains

Clock	Clocked Processes
adc_clk	p_adc_en, p_adc_capture
clk	p_ctrl, p_out

Dual-Clock Instances

The following instances connect clock-named ports to different signals, indicating an asynchronous FIFO, dual-port RAM, or similar CDC structure.

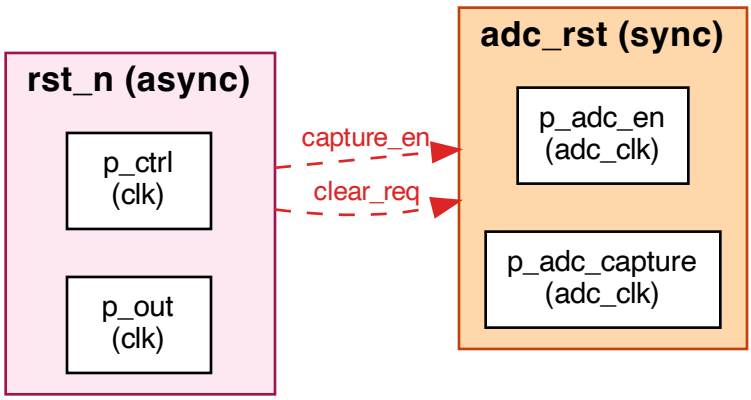
Instance	Component	Clock Port Connections
u_fifo	async_fifo	wr_clk → adc_clk, rd_clk → clk

Signal Crossings

Signal	Source Domain	Destination Domain	Synchronized
capture_en	clk	adc_clk	Yes *(two-flop)*
clear_req	clk	adc_clk	Yes *(two-flop)*

2.8.5.7 sample_capture — Reset Domain Analysis

Auto-extracted from `sample_capture.sv`.



Reset Domains

Process	Clock	Reset	Style
`p_adc_en`	`adc_clk`	`adc_rst`	Sync
`p_adc_capture`	`adc_clk`	`adc_rst`	Sync
`p_ctrl`	`clk`	`rst_n`	Async
`p_out`	`clk`	`rst_n`	Async

Warning

The following signals cross between different reset domains. Logic driven under one reset may not be in a consistent state when the other reset releases.

Signal Crossings

Signal	Source Reset	Destination Reset
`capture_en`	`rst_n`	`adc_rst`
`clear_req`	`rst_n`	`adc_rst`

2.8.5.8 Submodules

async_fifo

async_fifo — Entity

Source file: `src/async_fifo.vhd`.

ENTITY **async_fifo** IS

Asynchronous FIFO.

Moves words from the write clock domain to the read clock domain. Each side keeps a binary pointer for addressing and a Gray-coded copy that crosses to the other domain through a two-flop synchroniser, so only one bit changes at a time and a pointer is never sampled half-updated.

Depth is 2**DEPTH_LOG2 words. Full and empty are conservative: each side sees the other side's pointer a cycle or two late, so a flag may stay asserted slightly longer than needed, but is never released early. **Generics**

Generic	Type	Default	Description
WIDTH	positive	12	Width of one stored word in bits (default matches full_pkg::SAMPLE_W).
DEPTH_LOG2	positive	4	Base-2 logarithm of the number of words stored.

Ports

Port	Type	Mode	Description
wr_clk	std_logic	in	Write-side clock.
wr_rst_n	std_logic	in	Write-side asynchronous active-low reset.
wr_en	std_logic	in	Write strobe; ignored while wr_full is high.
wr_data	std_logic_vector(WIDTH - 1 downto 0)	in	Word to store.
wr_full	std_logic	out	High when no further word can be accepted.
rd_clk	std_logic	in	Read-side clock.
rd_rst_n	std_logic	in	Read-side asynchronous active-low reset.
rd_en	std_logic	in	Read strobe; ignored while rd_empty is high.
rd_data	std_logic_vector(WIDTH - 1 downto 0)	out	Oldest stored word, valid while rd_empty is low.
rd_empty	std_logic	out	High when there is nothing to read.

Annotated Source

Listing 9: src/async_fifo.vhd

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  -- Asynchronous FIFO.
6  --
7  -- Moves words from the write clock domain to the read clock domain. Each side
8  -- keeps a binary pointer for addressing and a Gray-coded copy that crosses to
9  -- the other domain through a two-flop synchroniser, so only one bit changes at
10 -- a time and a pointer is never sampled half-updated.
11 --
```

(continues on next page)

(continued from previous page)

```
-- Depth is 2**DEPTH_LOG2 words. Full and empty are conservative: each side
-- sees the other side's pointer a cycle or two late, so a flag may stay
-- asserted slightly longer than needed, but is never released early.
entity async_fifo is
    generic (
        -- Width of one stored word in bits (default matches full_pkg::SAMPLE_W).
        WIDTH      : positive := 12;
        -- Base-2 logarithm of the number of words stored.
        DEPTH_LOG2 : positive := 4
    );
    port (
        -- Write-side clock.
        wr_clk   : in  std_logic;
        -- Write-side asynchronous active-low reset.
        wr_rst_n : in  std_logic;
        -- Write strobe; ignored while wr_full is high.
        wr_en    : in  std_logic;
        -- Word to store.
        wr_data  : in  std_logic_vector(WIDTH - 1 downto 0);
        -- High when no further word can be accepted.
        wr_full  : out std_logic;
        -- Read-side clock.
        rd_clk   : in  std_logic;
        -- Read-side asynchronous active-low reset.
        rd_rst_n : in  std_logic;
        -- Read strobe; ignored while rd_empty is high.
        rd_en    : in  std_logic;
        -- Oldest stored word, valid while rd_empty is low.
        rd_data  : out std_logic_vector(WIDTH - 1 downto 0);
        -- High when there is nothing to read.
        rd_empty : out std_logic
    );
end entity async_fifo;

architecture rtl of async_fifo is
    constant PTR_BITS : positive := DEPTH_LOG2 + 1;

    type t_mem is array (0 to 2 ** DEPTH_LOG2 - 1) of std_logic_vector(WIDTH - 1 downto␣
↪0);
    signal mem : t_mem;

    -- Write domain: binary and Gray write pointers, then the read pointer
    -- as seen through its synchroniser.
    signal wptr_bin   : unsigned(PTR_BITS - 1 downto 0) := (others => '0');
    signal wptr_gray  : std_logic_vector(PTR_BITS - 1 downto 0) := (others => '0');
    signal rptr_meta  : std_logic_vector(PTR_BITS - 1 downto 0) := (others => '0');
    signal rptr_wsync : std_logic_vector(PTR_BITS - 1 downto 0) := (others => '0');

    -- Read domain: the mirror image.
    signal rptr_bin   : unsigned(PTR_BITS - 1 downto 0) := (others => '0');
    signal rptr_gray  : std_logic_vector(PTR_BITS - 1 downto 0) := (others => '0');
    signal wptr_meta  : std_logic_vector(PTR_BITS - 1 downto 0) := (others => '0');
```

(continues on next page)

(continued from previous page)

```
    signal wptr_rsync : std_logic_vector(PTR_BITS - 1 downto 0) := (others => '0');

    signal wfull  : std_logic;
    signal rempty : std_logic;

    function to_gray(b : unsigned) return std_logic_vector is
    begin
        return std_logic_vector(b xor shift_right(b, 1));
    end function to_gray;
begin

    -- p_wr_ptr: Stores a word and advances the write pointer.
    p_wr_ptr : process(wr_clk, wr_rst_n) is
        variable next_bin : unsigned(PTR_BITS - 1 downto 0);
    begin
        if wr_rst_n = '0' then
            wptr_bin  <= (others => '0');
            wptr_gray <= (others => '0');
        elsif rising_edge(wr_clk) then
            next_bin := wptr_bin;
            if wr_en = '1' and wfull = '0' then
                mem(to_integer(wptr_bin(DEPTH_LOG2 - 1 downto 0))) <= wr_data;
                next_bin := wptr_bin + 1;
            end if;
            wptr_bin  <= next_bin;
            wptr_gray <= to_gray(next_bin);
        end if;
    end process p_wr_ptr;

    -- p_rptr_sync: Brings the read pointer into the write domain.
    p_rptr_sync : process(wr_clk, wr_rst_n) is
    begin
        if wr_rst_n = '0' then
            rptr_meta  <= (others => '0');
            rptr_wsync <= (others => '0');
        elsif rising_edge(wr_clk) then
            rptr_meta  <= rptr_gray;
            rptr_wsync <= rptr_meta;
        end if;
    end process p_rptr_sync;

    -- p_rd_ptr: Advances the read pointer after a word is taken.
    p_rd_ptr : process(rd_clk, rd_rst_n) is
        variable next_bin : unsigned(PTR_BITS - 1 downto 0);
    begin
        if rd_rst_n = '0' then
            rptr_bin  <= (others => '0');
            rptr_gray <= (others => '0');
        elsif rising_edge(rd_clk) then
            next_bin := rptr_bin;
            if rd_en = '1' and rempty = '0' then
                next_bin := rptr_bin + 1;
```

(continues on next page)

(continued from previous page)

```
            end if;
            rptr_bin  <= next_bin;
            rptr_gray <= to_gray(next_bin);
        end if;
    end process p_rd_ptr;

    -- p_wptr_sync: Brings the write pointer into the read domain.
    p_wptr_sync : process(rd_clk, rd_rst_n) is
    begin
        if rd_rst_n = '0' then
            wptr_meta  <= (others => '0');
            wptr_rsync <= (others => '0');
        elsif rising_edge(rd_clk) then
            wptr_meta  <= wptr_gray;
            wptr_rsync <= wptr_meta;
        end if;
    end process p_wptr_sync;

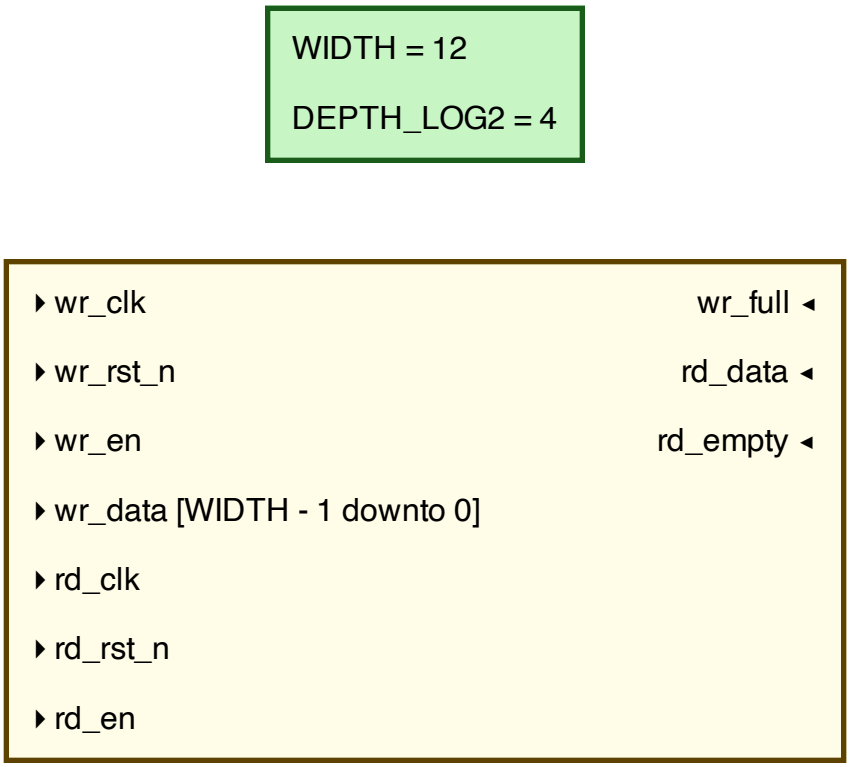
    -- Full: the write Gray pointer equals the synchronised read pointer with
    -- its two top bits inverted.
    wfull <= '1' when wptr_gray = (not rptr_wsync(PTR_BITS - 1 downto PTR_BITS - 2)
                                   & rptr_wsync(PTR_BITS - 3 downto 0)) else '0';
    -- Empty: both Gray pointers are equal.
    rempty <= '1' when rptr_gray = wptr_rsync else '0';

    rd_data  <= mem(to_integer(rptr_bin(DEPTH_LOG2 - 1 downto 0)));
    wr_full  <= wfull;
    rd_empty <= rempty;

end architecture rtl;
```

async_fifo — Block Diagram

Auto-extracted from `async_fifo.vhd`.



Ports

Port	Direction	Type	Description
`wr_clk`	`in`	`std_logic`	Write-side clock.
`wr_rst_n`	`in`	`std_logic`	Write-side asynchronous active-low reset.
`wr_en`	`in`	`std_logic`	Write strobe; ignored while wr_full is high.
`wr_data`	`in`	`std_logic_vector(WIDTH - 1 downto 0)`	Word to store.
`wr_full`	`out`	`std_logic`	High when no further word can be accepted.
`rd_clk`	`in`	`std_logic`	Read-side clock.
`rd_rst_n`	`in`	`std_logic`	Read-side asynchronous active-low reset.
`rd_en`	`in`	`std_logic`	Read strobe; ignored while rd_empty is high.
`rd_data`	`out`	`std_logic_vector(WIDTH - 1 downto 0)`	Oldest stored word, valid while rd_empty is low.
`rd_empty`	`out`	`std_logic`	High when there is nothing to read.

Generics

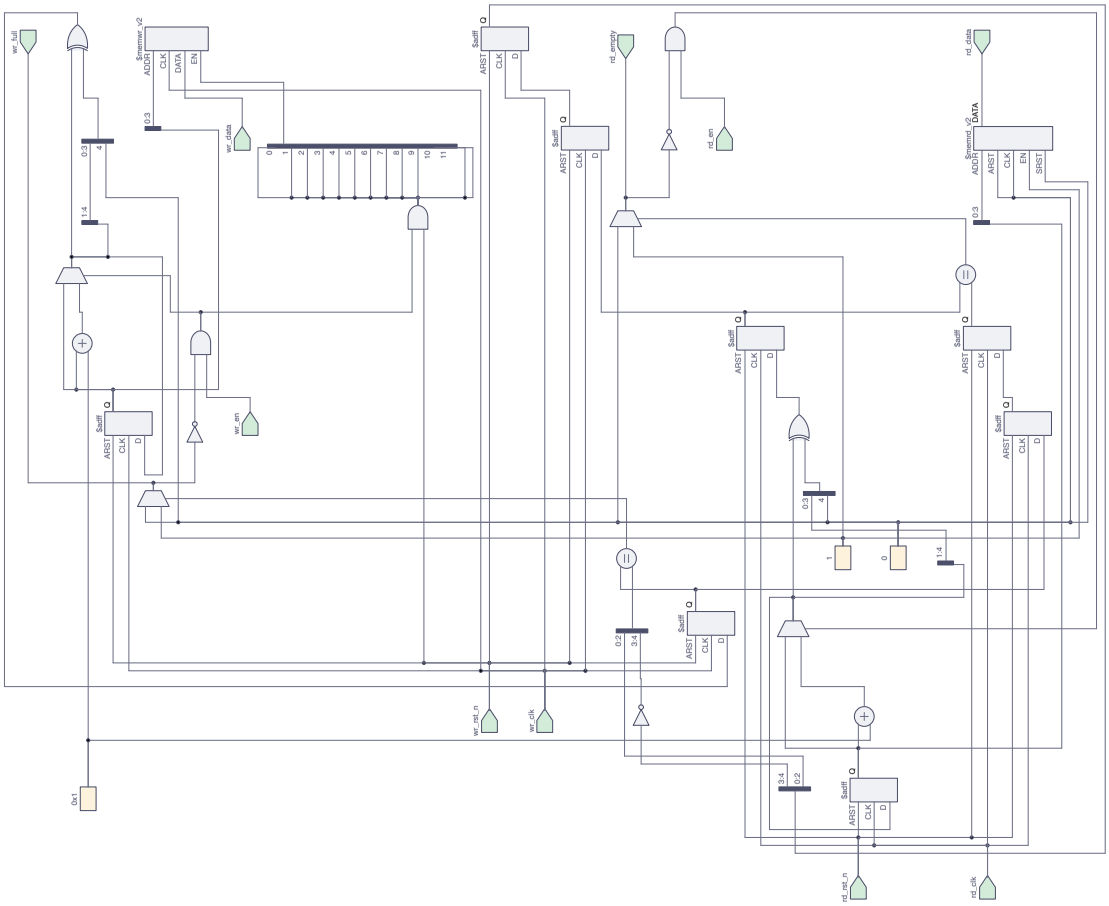
Name	Type	Default	Description
WIDTH	positive	12	Width of one stored word in bits (default matches full_pkg::SAMPLE_W).
DEPTH_LOG2	positive	4	Base-2 logarithm of the number of words stored.

Signals

Signal	Type	Description
mem	t_mem	—
wptr_bin	unsigned(PTR_BITS - 1 downto 0)	as seen through its synchroniser.
wptr_gray	std_logic_vector(PTR_BITS - 1 downto 0)	—
rptr_meta	std_logic_vector(PTR_BITS - 1 downto 0)	—
rptr_wsync	std_logic_vector(PTR_BITS - 1 downto 0)	—
rptr_bin	unsigned(PTR_BITS - 1 downto 0)	Read domain: the mirror image.
rptr_gray	std_logic_vector(PTR_BITS - 1 downto 0)	—
wptr_meta	std_logic_vector(PTR_BITS - 1 downto 0)	—
wptr_rsync	std_logic_vector(PTR_BITS - 1 downto 0)	—
wfull	std_logic	—
rempty	std_logic	—

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


wr_full
$memmw_v2
ADDR
CLK
DATA
EN
wr_data
$adff
ARST
CLK
D
rd_empty
rd_en
rd_data
$memmd_rd
SRST
wr_en
wr_rst_n
wr_clk
rd_rst_n
rd_clk
0x1

async_fifo — State Machine

No FSM detected in `async_fifo.vhd`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

async_fifo — Timing Diagrams

All timing diagrams extracted from `async_fifo.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

Processes

Auto-extracted from `async_fifo.vhd`.

p_wr_ptr

Source file: `async_fifo.vhd`
Source line: 75
Block type: `process`
Sensitivity list: `wr_clk`, `wr_rst_n`

Description

p_wr_ptr: Stores a word and advances the write pointer.

Source

```
p_wr_ptr : process(wr_clk, wr_rst_n) is
    variable next_bin : unsigned(PTR_BITS - 1 downto 0);
begin
    if wr_rst_n = '0' then
        wptr_bin  <= (others => '0');
        wptr_gray <= (others => '0');
    elsif rising_edge(wr_clk) then
        next_bin := wptr_bin;
        if wr_en = '1' and wfull = '0' then
            mem(to_integer(wptr_bin(DEPTH_LOG2 - 1 downto 0))) <= wr_data;
            next_bin := wptr_bin + 1;
        end if;
        wptr_bin  <= next_bin;
        wptr_gray <= to_gray(next_bin);
    end if;
end process p_wr_ptr;
```

p_rptr_sync

Source file: async_fifo.vhd
Source line: 93
Block type: process
Sensitivity list: wr_clk, wr_rst_n

Description

p_rptr_sync: Brings the read pointer into the write domain.

Source

```
p_rptr_sync : process(wr_clk, wr_rst_n) is
begin
    if wr_rst_n = '0' then
        rptr_meta  <= (others => '0');
        rptr_wsync <= (others => '0');
    elsif rising_edge(wr_clk) then
        rptr_meta  <= rptr_gray;
        rptr_wsync <= rptr_meta;
    end if;
end process p_rptr_sync;
```

p_rd_ptr

Source file: async_fifo.vhd
Source line: 105
Block type: process
Sensitivity list: rd_clk, rd_rst_n

Description

p_rd_ptr: Advances the read pointer after a word is taken.

Source

```
p_rd_ptr : process(rd_clk, rd_rst_n) is
    variable next_bin : unsigned(PTR_BITS - 1 downto 0);
begin
    if rd_rst_n = '0' then
        rptr_bin  <= (others => '0');
        rptr_gray <= (others => '0');
    elsif rising_edge(rd_clk) then
        next_bin := rptr_bin;
        if rd_en = '1' and rempty = '0' then
            next_bin := rptr_bin + 1;
        end if;
        rptr_bin  <= next_bin;
        rptr_gray <= to_gray(next_bin);
```

(continues on next page)

(continued from previous page)

```
    end if;
end process p_rd_ptr;
```

p_wptr_sync

Source file: `async_fifo.vhd`
Source line: 122
Block type: `process`
Sensitivity list: `rd_clk`, `rd_rst_n`

Description

p_wptr_sync: Brings the write pointer into the read domain.

Source

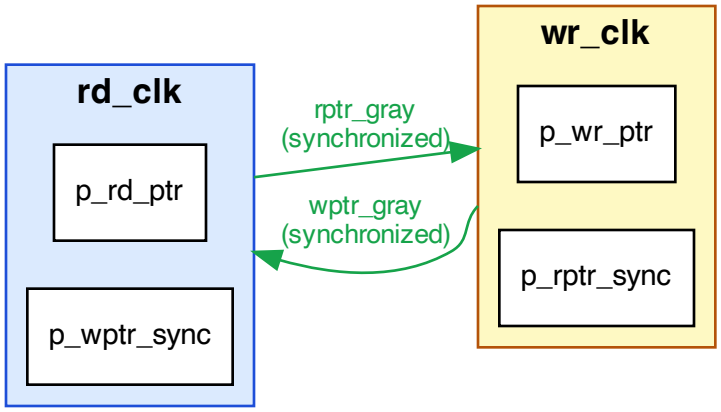
```
p_wptr_sync : process(rd_clk, rd_rst_n) is
begin
    if rd_rst_n = '0' then
        wptr_meta  <= (others => '0');
        wptr_rsync <= (others => '0');
    elsif rising_edge(rd_clk) then
        wptr_meta  <= wptr_gray;
        wptr_rsync <= wptr_meta;
    end if;
end process p_wptr_sync;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom
p_wr_ptr	`process`	`wr_clk`, `wr_rst_n`	75	—
p_rptr_sync	`process`	`wr_clk`, `wr_rst_n`	93	—
p_rd_ptr	`process`	`rd_clk`, `rd_rst_n`	105	—
p_wptr_sync	`process`	`rd_clk`, `rd_rst_n`	122	—

async_fifo — Clock Domain Analysis

Auto-extracted from `async_fifo.vhd`.



Clock Domains

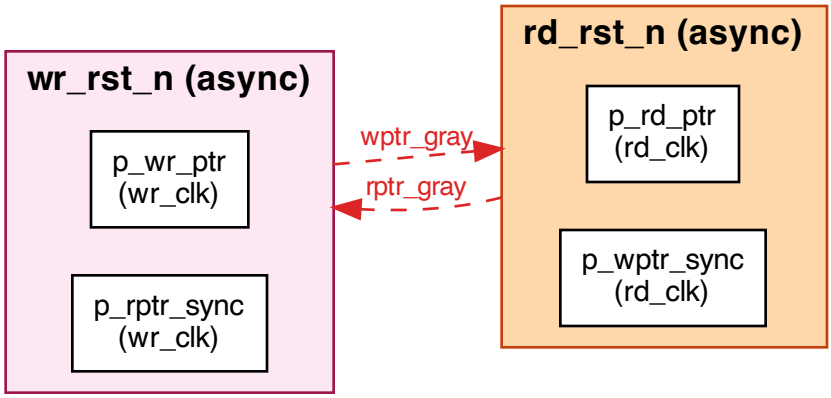
Clock	Clocked Processes
`rd_clk`	`p_rd_ptr`, `p_wptr_sync`
`wr_clk`	`p_wr_ptr`, `p_rptr_sync`

Signal Crossings

Signal	Source Domain	Destination Domain	Synchronized
`rptr_gray`	`rd_clk`	`wr_clk`	Yes *(two-flop)*
`wptr_gray`	`wr_clk`	`rd_clk`	Yes *(two-flop)*

async_fifo — Reset Domain Analysis

Auto-extracted from `async_fifo.vhd`.



Reset Domains

Process	Clock	Reset	Style
`p_rd_ptr`	`rd_clk`	`rd_rst_n`	Async
`p_wptr_sync`	`rd_clk`	`rd_rst_n`	Async
`p_wr_ptr`	`wr_clk`	`wr_rst_n`	Async
`p_rptr_sync`	`wr_clk`	`wr_rst_n`	Async

Warning

The following signals cross between different reset domains. Logic driven under one reset may not be in a consistent state when the other reset releases.

Signal Crossings

Signal	Source Reset	Destination Reset
`rptr_gray`	`rd_rst_n`	`wr_rst_n`
`wptr_gray`	`wr_rst_n`	`rd_rst_n`

2.8.6 sample_ctrl

2.8.6.1 sample_ctrl — Entity

Source file: `src/sample_ctrl.vhd`.

ENTITY **`sample_ctrl`** IS

Sampler controller.

Runs one capture window per start request: it arms, enables the capture path for RUN_CYCLES clock cycles (or until the FIFO overflows), then reports done and waits for the request to be withdrawn before it can run again.

State sequence: IDLE -> ARMED -> RUN -> DONE -> IDLE. **Generics**

Generic	Type	Default	Description
RUN_CYCLES	positive	1024	Length of the capture window in clock cycles.

Ports

Port	Type	Mode	Description
clk	std_logic	in	System clock, rising-edge triggered.
rst	std_logic	in	Synchronous active-high reset. Returns the FSM to IDLE.
start	std_logic	in	Start request; held high until the capture is done.
overflow	std_logic	in	A sample was lost; ends the window early.
capture_en	std_logic	out	High while samples should be captured.
busy	std_logic	out	High from arming until the request is withdrawn.
state_code	std_logic_vector(1 downto 0)	out	Current state, 0 IDLE, 1 ARMED, 2 RUN, 3 DONE.

Annotated Source

Listing 10: src/sample_ctrl.vhd

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  -- Sampler controller.
5  --
6  -- Runs one capture window per start request: it arms, enables the capture
7  -- path for RUN_CYCLES clock cycles (or until the FIFO overflows), then reports
8  -- done and waits for the request to be withdrawn before it can run again.
9  --
10 -- State sequence: IDLE -> ARMED -> RUN -> DONE -> IDLE.
11 entity sample_ctrl is
12     generic (
13         -- Length of the capture window in clock cycles.
14         RUN_CYCLES : positive := 1024
15     );
16     port (
17         -- System clock, rising-edge triggered.
18         clk        : in  std_logic;
19         -- Synchronous active-high reset. Returns the FSM to IDLE.
20         rst        : in  std_logic;
21         -- Start request; held high until the capture is done.
22         start      : in  std_logic;
23         -- A sample was lost; ends the window early.
24         overflow   : in  std_logic;
25         -- High while samples should be captured.
```

(continues on next page)

(continued from previous page)

```
        capture_en : out std_logic;
        -- High from arming until the request is withdrawn.
        busy       : out std_logic;
        -- Current state, 0 IDLE, 1 ARMED, 2 RUN, 3 DONE.
        state_code : out std_logic_vector(1 downto 0)
    );
end entity sample_ctrl;

architecture rtl of sample_ctrl is

    -- FSM state type for the capture window.
    type t_state is (
        -- Waiting for a start request.
        IDLE,
        -- Request seen; capture path is being enabled.
        ARMED,
        -- Capturing samples.
        RUN,
        -- Window over; waiting for the request to drop.
        DONE
    );

    signal state      : t_state := IDLE;
    signal next_state : t_state;
    -- Cycles spent in RUN.
    signal run_cnt    : natural range 0 to RUN_CYCLES - 1 := 0;
    -- High on the last cycle of the window.
    signal run_last   : std_logic;

begin

    run_last <= '1' when run_cnt = RUN_CYCLES - 1 else '0';

    -- p_state_reg: Clocked state register and window counter.
    --
    -- Synchronous reset returns the state to IDLE and clears the counter.
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "clk",   "wave": "P......." },
    --      { "name": "rst",   "wave": "1.0....." },
    --      { "name": "start", "wave": "0..1...." },
    --      { "name": "state", "wave": "=.=.==..", "data": ["(any)","IDLE","ARMED","RUN
↪"] }
    --    ]}
    p_state_reg : process(clk) is
    begin
        if rising_edge(clk) then
            if rst = '1' then
                state   <= IDLE;
                run_cnt <= 0;
```

(continues on next page)

(continued from previous page)

```
            else
                state <= next_state;
                if state = RUN then
                    if run_cnt = RUN_CYCLES - 1 then
                        run_cnt <= 0;
                    else
                        run_cnt <= run_cnt + 1;
                    end if;
                else
                    run_cnt <= 0;
                end if;
            end if;
        end if;
    end process p_state_reg;

    -- p_next_state: Combinational next-state logic.
    --
    -- A start request arms the controller; the window ends after RUN_CYCLES
    -- cycles or on overflow; DONE is left when the request is withdrawn.
    --
    -- .. mermaid::
    --    :caption: Controller states
    --
    --    stateDiagram-v2
    --      [*] --> IDLE
    --      IDLE --> ARMED: start
    --      ARMED --> RUN
    --      RUN --> DONE: window over or overflow
    --      DONE --> IDLE: start withdrawn
    --
    -- .. wavedrom::
    --
    --    { "signal": [
    --      { "name": "clk",      "wave": "P........." },
    --      { "name": "start",    "wave": "01.....0.." },
    --      { "name": "run_last", "wave": "0....10..." },
    --      { "name": "state",    "wave": "=.==..=.=.", "data": ["IDLE","ARMED","RUN",
↪"DONE","IDLE"] }
    --    ]}
    p_next_state : process(state, start, overflow, run_last) is
    begin
        next_state <= state;

        case state is
            when IDLE =>
                if start = '1' then
                    next_state <= ARMED;
                end if;

            when ARMED =>
                next_state <= RUN;

```

(continues on next page)

(continued from previous page)

```
            when RUN =>
                if run_last = '1' then
                    next_state <= DONE;
                elsif overflow = '1' then
                    next_state <= DONE;
                end if;

            when DONE =>
                if start = '0' then
                    next_state <= IDLE;
                end if;
        end case;
    end process p_next_state;

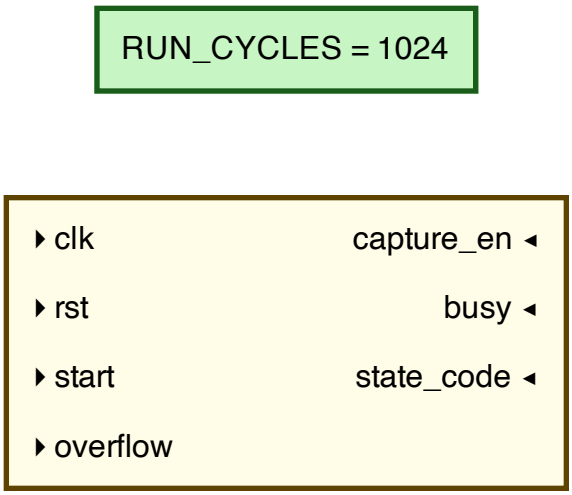
    -- p_outputs: Combinational Moore output decode.
    --
    -- Capture is enabled in RUN only; busy covers everything but IDLE.
    p_outputs : process(state) is
    begin
        capture_en <= '0';
        busy       <= '1';
        state_code <= "00";

        case state is
            when IDLE  => busy <= '0';
            when ARMED => state_code <= "01";
            when RUN   => capture_en <= '1'; state_code <= "10";
            when DONE  => state_code <= "11";
        end case;
    end process p_outputs;

end architecture rtl;
```

2.8.6.2 sample_ctrl — Block Diagram

Auto-extracted from `sample_ctrl.vhd`.



Ports

Port	Direction	Type	Description
`clk`	`in`	`std_logic`	System clock, rising-edge triggered.
`rst`	`in`	`std_logic`	Synchronous active-high reset. Returns the FSM to IDLE.
`start`	`in`	`std_logic`	Start request; held high until the capture is done.
`overflow`	`in`	`std_logic`	A sample was lost; ends the window early.
`capture_en`	`out`	`std_logic`	High while samples should be captured.
`busy`	`out`	`std_logic`	High from arming until the request is withdrawn.
`state_code`	`out`	`std_logic_vector(1 downto 0)`	Current state, 0 IDLE, 1 ARMED, 2 RUN, 3 DONE.

Generics

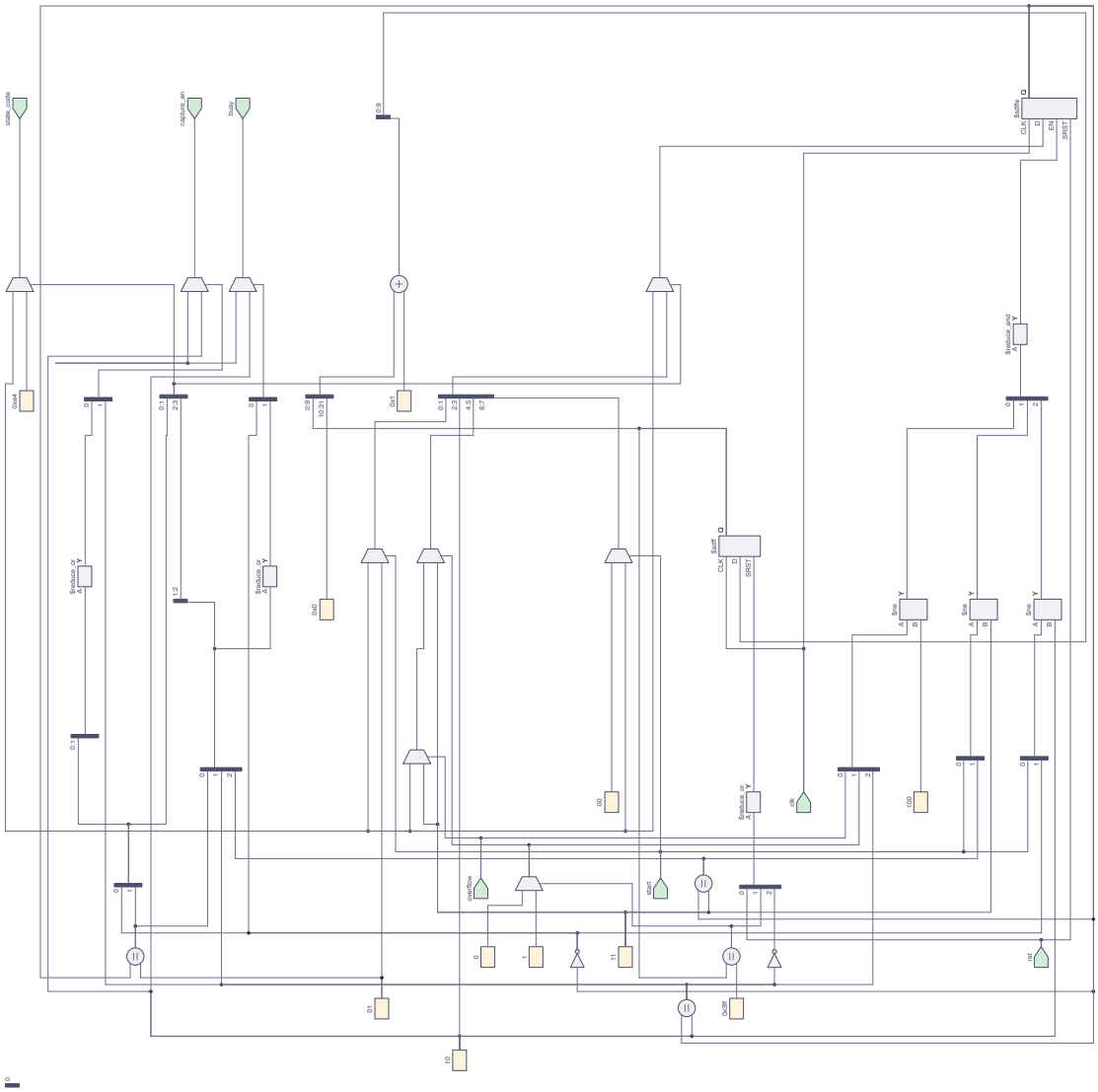
Name	Type	Default	Description
`RUN_CYCLES`	`positive`	`1024`	Length of the capture window in clock cycles.

Signals

Signal	Type	Description
state	t_state	—
next_state	t_state	—
run_cnt	natural	Cycles spent in RUN.
run_last	std_logic	High on the last cycle of the window.

RTL Detail

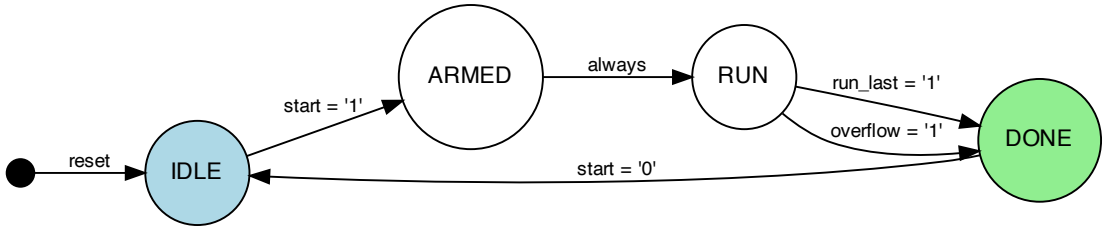
Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


state_code
capture_en
busy
0:9
$auto$ff
CLK
D
EN
SRST
$add
0x1
$reduce_and
A
Y
$reduce_or
$eq
B
0x0
eq
0x3f
overflow
start
10
01
00
11
100
0

2.8.6.3 sample_ctrl State Machine

Auto-extracted from `sample_ctrl.vhd`.

State Transition Diagram



State Output Table

State	capture_en	busy
IDLE	0	0
ARMED	0	1
RUN	1	1
DONE	0	1

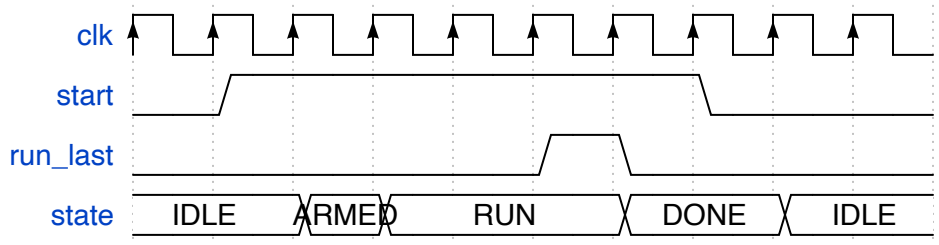
Transitions

From	To	Condition
IDLE	ARMED	start = '1'
ARMED	RUN	(always)
RUN	DONE	run_last = '1'
RUN	DONE	overflow = '1'
DONE	IDLE	start = '0'

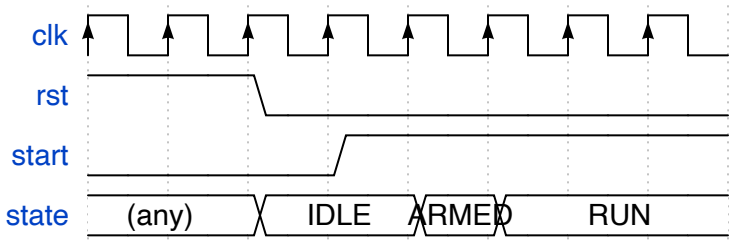
2.8.6.4 sample_ctrl — Timing Diagrams

All timing diagrams extracted from `sample_ctrl.vhd`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

P Next State



P State Reg



2.8.6.5 Processes

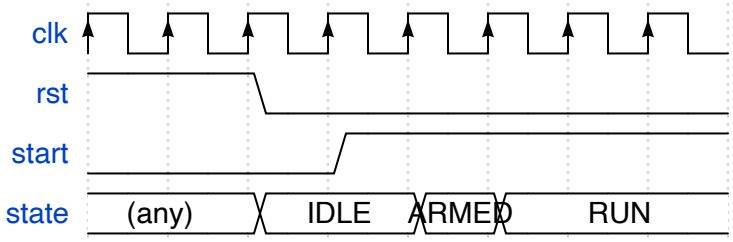
Auto-extracted from `sample_ctrl.vhd`.

p_state_reg

Source file: `sample_ctrl.vhd`
Source line: 71
Block type: `process`
Sensitivity list: `clk`

Description

p_state_reg: Clocked state register and window counter. Synchronous reset returns the state to IDLE and clears the counter.



Source

```
p_state_reg : process(clk) is
begin
    if rising_edge(clk) then
        if rst = '1' then
            state   <= IDLE;
            run_cnt <= 0;
        else
            state <= next_state;
            if state = RUN then
                if run_cnt = RUN_CYCLES - 1 then
                    run_cnt <= 0;
                else
                    run_cnt <= run_cnt + 1;
                end if;
            else
```

(continues on next page)

(continued from previous page)

```
                run_cnt <= 0;
            end if;
        end if;
    end if;
end process p_state_reg;
```

p_next_state

Source file: sample_ctrl.vhd
Source line: 115
Block type: process
Sensitivity list: state, start, overflow, run_last

Description

p_next_state: Combinational next-state logic. A start request arms the controller; the window ends after RUN_CYCLES cycles or on overflow; DONE is left when the request is withdrawn.

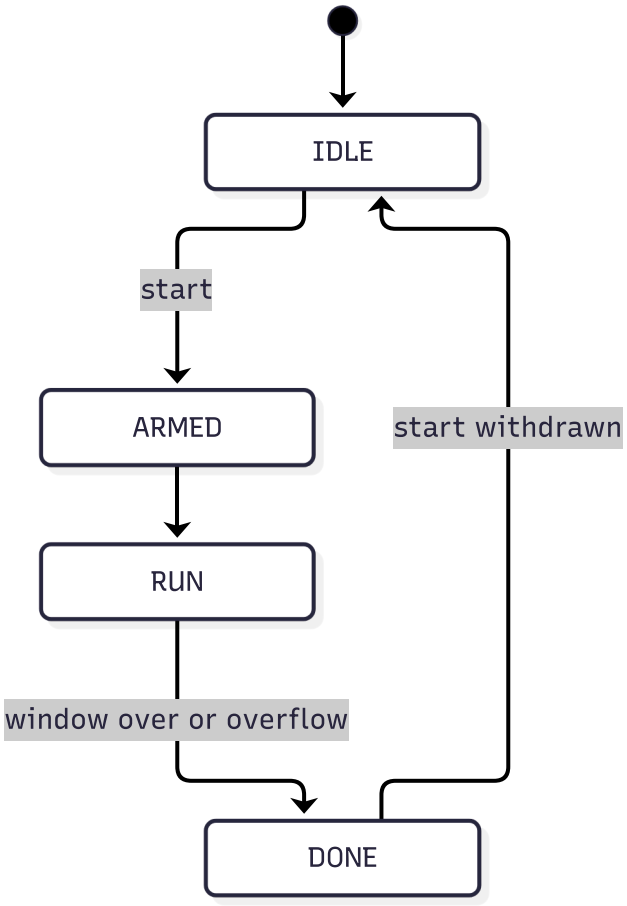
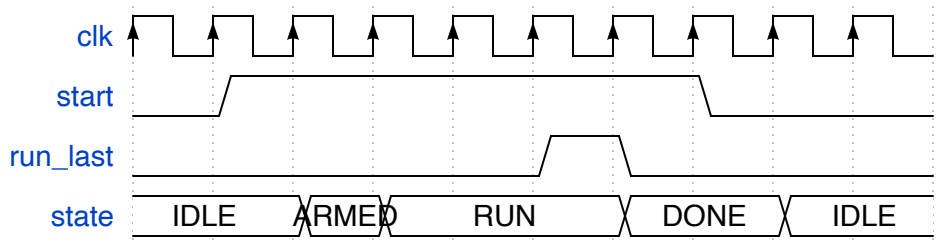


Fig. 5: Controller states



Signal Assignments

State	next_state
IDLE	ARMED
ARMED	RUN
RUN	DONE
DONE	IDLE

Source

```
p_next_state : process(state, start, overflow, run_last) is
begin
    next_state <= state;

    case state is
        when IDLE =>
            if start = '1' then
                next_state <= ARMED;
            end if;

        when ARMED =>
            next_state <= RUN;

        when RUN =>
            if run_last = '1' then
                next_state <= DONE;
            elsif overflow = '1' then
                next_state <= DONE;
            end if;

        when DONE =>
            if start = '0' then
                next_state <= IDLE;
            end if;
    end case;
end process p_next_state;
```

p_outputs

Source file: sample_ctrl.vhd
Source line: 145
Block type: process

Sensitivity list: `state`

Description

p_outputs: Combinational Moore output decode. Capture is enabled in RUN only; busy covers everything but IDLE.

Source

```
p_outputs : process(state) is
begin
    capture_en <= '0';
    busy       <= '1';
    state_code <= "00";

    case state is
        when IDLE  => busy <= '0';
        when ARMED => state_code <= "01";
        when RUN   => capture_en <= '1'; state_code <= "10";
        when DONE  => state_code <= "11";
    end case;
end process p_outputs;
```

Summary

Block	Type	Sensitivity	Source Line	WaveDrom	Mermaid
p_state_reg	process	clk	71	✓	—
p_next_state	process	state, start, overflow, run_last	115	✓	✓
p_outputs	process	state	145	—	—

2.8.6.6 sample_ctrl — Clock Domain Analysis

Auto-extracted from `sample_ctrl.vhd`.

Note

No clock domain crossings detected. All clocked logic uses a single clock domain (`clk`).

Clock Domain

Clock	Clocked Processes
clk	p_state_reg

2.8.6.7 sample_ctrl — Reset Domain Analysis

Auto-extracted from `sample_ctrl.vhd`.

Note

All clocked processes use a single reset domain (`rst`, sync). No reset domain crossings detected.

Reset Domain

Process	Clock	Reset	Style
`p_state_reg`	`clk`	`rst`	Sync

2.8.7 sample_stream

2.8.7.1 sample_stream — Entity

Source file: `src/sample_stream.sv`.

Captured samples as a filtered byte stream.

Sends the low byte of each sample as a one-byte packet and runs it through a stream_filter, so only samples whose low byte reaches the threshold leave on the flat AXI4-Stream master port.

Annotated Source

Listing 11: src/sample_stream.sv

```
// Captured samples as a filtered byte stream.
//
// Sends the low byte of each sample as a one-byte packet and runs it through a
// stream_filter, so only samples whose low byte reaches the threshold leave on
// the flat AXI4-Stream master port.
module sample_stream (
    // System clock.
    input  logic                        clk,
    // Synchronous active-high reset.
    input  logic                        rst,
    // Filtering is on while high.
    input  logic                        enable,
    // Smallest low-byte value that lets a sample through.
    input  logic [7:0]                  threshold,
    // Sample and its handshake from the capture path.
    input  logic [full_pkg::SAMPLE_W-1:0] sample_data,
    input  logic                        sample_valid,
    output logic                        sample_ready,
    // Forwarded bytes.
    output logic [7:0]                  m_axis_tdata,
    output logic                        m_axis_tvalid,
    input  logic                        m_axis_tready,
    output logic                        m_axis_tlast
);
    axis_if s_if ();
    axis_if m_if ();
    logic [15:0] unused_count;

```

(continues on next page)

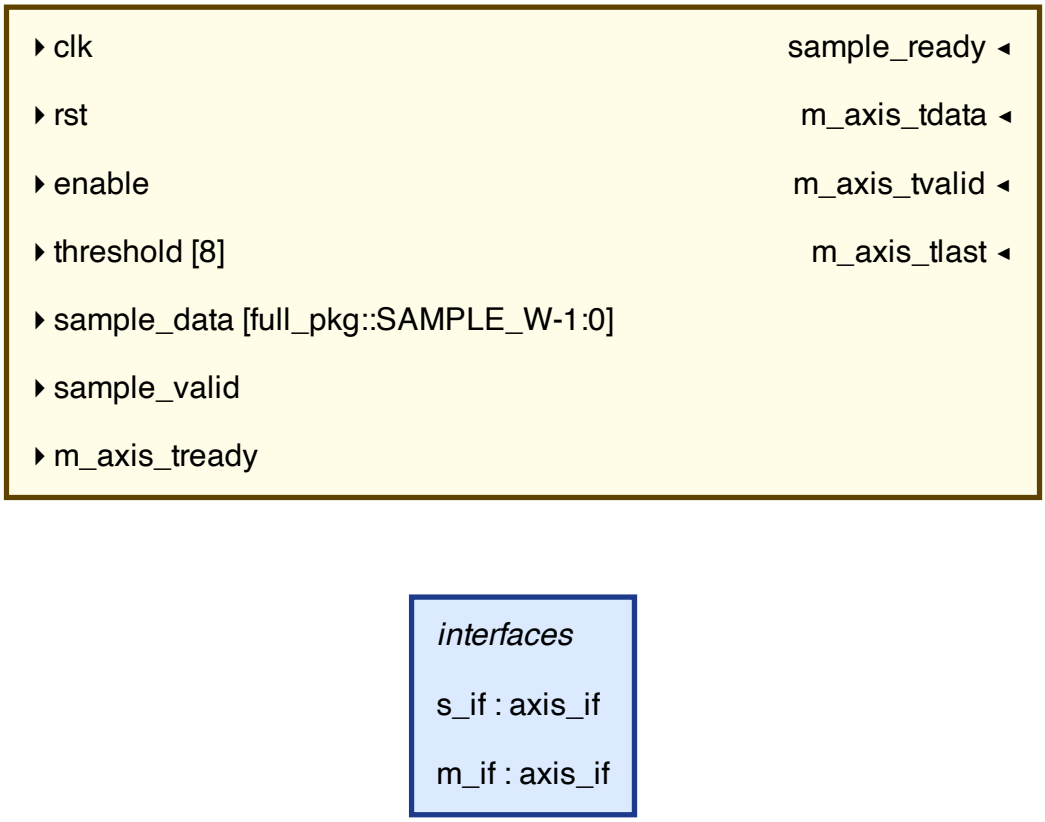
(continued from previous page)

```
    assign s_if.tdata  = sample_data[7:0];
    assign s_if.tvalid = sample_valid;
    assign s_if.tlast  = 1'b1;
    assign sample_ready = s_if.tready;
    assign m_axis_tdata  = m_if.tdata;
    assign m_axis_tvalid = m_if.tvalid;
    assign m_axis_tlast  = m_if.tlast;
    assign m_if.tready   = m_axis_tready;

    stream_filter u_filter (
        .clk(clk), .rst(rst), .enable(enable), .clear(1'b0), .threshold(threshold),
        .s_axis(s_if), .m_axis(m_if), .count(unused_count)
    );
endmodule
```

2.8.7.2 sample_stream — Block Diagram

Auto-extracted from `sample_stream.sv`.



Ports

Interfaces

`axis_if` — instantiated here as `s_if`, `m_if`.

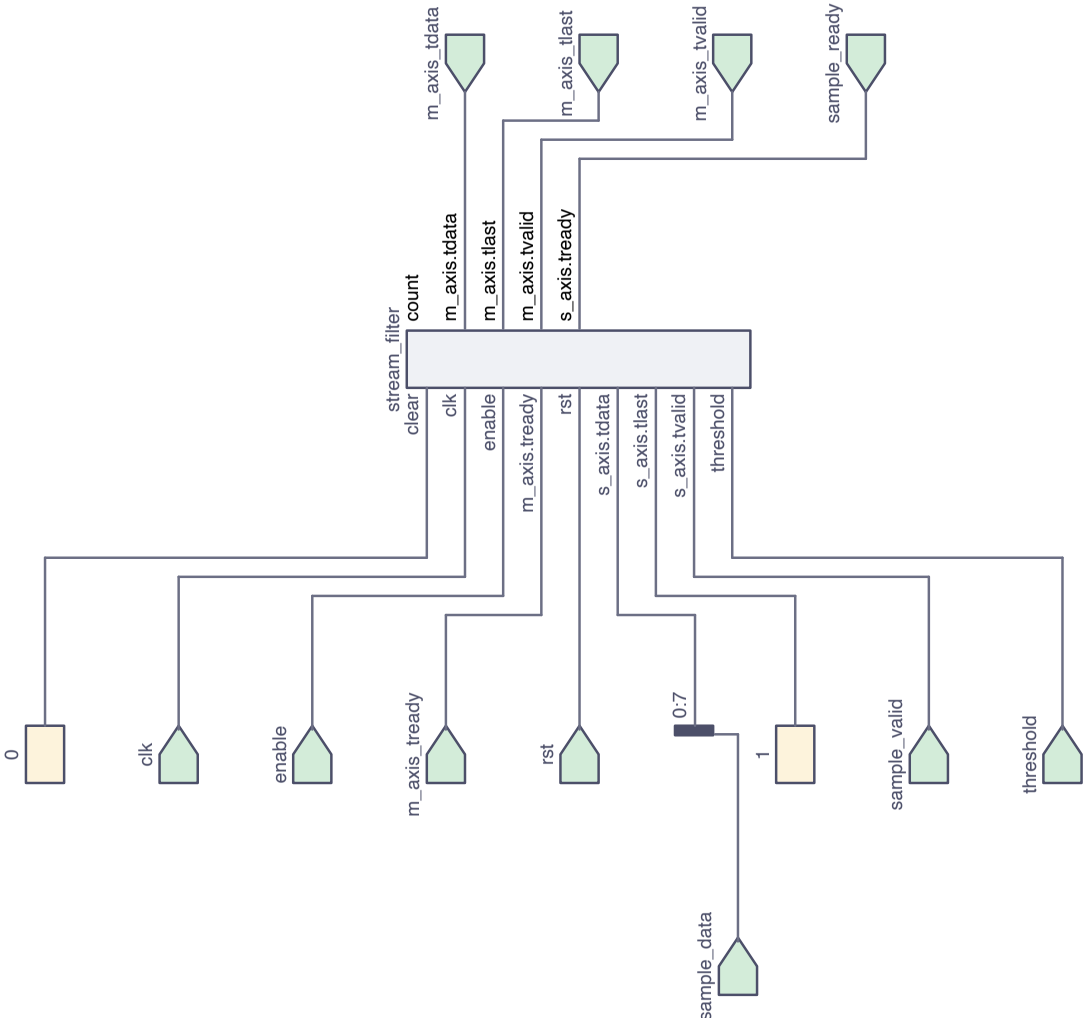
Signal	Type	master	slave	Description
`tdata`	`logic[7:0]`	out	in	Payload byte.
`tvalid`	`logic`	out	in	Master has a byte; slave takes it when tready is also high.
`tready`	`logic`	in	out	Slave can take a byte this cycle.
`tlast`	`logic`	out	in	Last byte of a packet.

Signals

Signal	Type	Description
`unused_count`	`logic[15:0]`	—

RTL Detail

Gate-level netlist generated by `yosys` and rendered with `netlistsvg`.


m_axis_tdata
m_axis_tlast
m_axis_tvalid
sample_ready
stream_filter
count
m_axis.tdata
m_axis.tlast
m_axis.tvalid
s_axis.tready
clear
clk
enable
m_axis.tready
rst
s_axis.tdata
s_axis.tlast
s_axis.tvalid
threshold
0
clk
enable
m_axis_tready
rst
0:7
1
sample_valid
threshold
sample_data

2.8.7.3 sample_stream — State Machine

No FSM detected in `sample_stream.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.8.7.4 sample_stream — Timing Diagrams

All timing diagrams extracted from `sample_stream.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.8.7.5 sample_stream — Clock Domain Analysis

Auto-extracted from `sample_stream.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.8.7.6 sample_stream — Reset Domain Analysis

Auto-extracted from `sample_stream.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.8.7.7 Submodules

Note

Modules marked **shared** are instantiated by multiple parents. Their documentation appears once; all parents link to the same page.

Chapter 3

Register Maps

Chapter 4

Documentation Coverage

Chapter 5

Testbenches

Framework	Testbenches	Tests
VUnit	2	6
cocotb	1	3 (1 skipped)
Directed	3	6
Total	6	15 (1 skipped)

5.1 VUnit

5.1.1 tb_sample_ctrl

File
`tb/vunit/tb_sample_ctrl.vhd`

Tests module
sample_ctrl

Purpose
Testbench for sample_ctrl. Starts a capture and checks the controller arms, runs for its window, then reports done and waits for the request to be withdrawn.

Tests
3 tests

Test	Purpose
`capture_window_length`	The window opens after arming and lasts RUN_CYCLES cycles.
`idle_without_start`	Nothing runs without a start request.
`overflow_ends_window`	An overflow ends the window early.

5.1.2 tb_stream_filter

File
`tb/vunit/tb_stream_filter.sv`

Tests module
stream_filter

Purpose
Testbench for stream_filter. Sends single-byte and multi-byte packets and checks that a packet is forwarded or dropped as a whole according to its first byte.

Tests
3 tests

Test	Purpose
`forwards_packet_at_threshold`	A packet whose first byte reaches the threshold is forwarded.
`drops_packet_below_threshold`	A packet whose first byte is below the threshold is dropped whole.
`counts_forwarded_packets`	Only forwarded packets are counted.

5.2 cocotb

5.2.1 test_axil_adapter

File
`tb/cocotb/test_axil_adapter.py`

Tests module
axil_adapter

Purpose
Register file tests through axil_adapter.

Tests
3 tests, 1 skipped

Test	Purpose
`reset_values`	Every register reads back zero after reset.
`scratch_write_read`	A value written to the scratch register reads back unchanged.
`start_runs_a_capture` *(skipped)*	Setting ctrl.start moves the status to running (needs the full top level).

5.3 Directed

5.3.1 tb_axil_adapter

File
`tb/directed/tb_axil_adapter.sv`

Tests module
axil_adapter

Purpose
Directed testbench for the AXI4-Lite adapter. Writes and reads the scratch register through the flat AXI4-Lite port, with no verification framework: each scenario below is marked with a TEST comment.

Tests
2 tests

Test	Purpose
scratch_write_read	a value written to scratch reads back unchanged
threshold_output	writing the threshold register drives the threshold port

5.3.2 tb_smoke

File
tb/directed/tb_smoke.sv

Tests module
—

Purpose
Clock and reset smoke test. Checks only that a free-running clock toggles; it does not instantiate any design module, so it is not linked to one.

Tests
1 test

Test	Purpose
clock_toggles	the clock changes level

5.3.3 tb_top

File
tb/directed/tb_top.vhd

Tests module
top

Purpose
Directed testbench for the top level. Programs the filters through the APB port, then streams bytes through the external pipeline, with no verification framework: each scenario below is marked with a TEST comment.

Tests
3 tests

Test	Purpose
register_write	a write over APB reaches the scratch register
capture_runs	setting start raises led_busy
ext_filter_pass	a packet starting above the threshold reaches m_ext

Chapter 6

Build

6.1 Summary

Yosys	0.56+186
Device	ECP5 25k class
Reports generated	2026-11-01 20:56
Documented at	v3.12.0-26-g3652455

6.2 Utilization

6.2.1 Device utilisation (nextpnr)

Resource	Used	Available	%
TRELLIS_IO	127	197	64
DCCA	2	56	3
TRELLIS_FF	300	24,288	1
TRELLIS_COMB	290	24,288	1
TRELLIS_RAMW	2	3,036	0

23 other resource types unused.

6.2.2 By module (Vivado)

Module	LUTs / 20,800	FFs / 41,600	BRAMs	DSPs
top	387	294	2	0
└ apb	52	70	0	0
└ capture	89	72	2	0
└ ctrl	47	38	0	0
└ ext	72	30	0	0
└ regs	109	72	0	0

6.3 Timing — Vivado

Clock	Target (MHz)	Fmax (MHz)	Period (ns)	WNS (ns)
clk	100.0	139.80	10.000	+2.847
adc_clk	71.4	89.66	14.000	+2.847

6.4 Warnings

184 warnings in the build logs; the most frequent:

Message	Count
IO '…' is unconstrained in LPF and will be automatically placed	126
Identifier '…' is implicitly declared.	57
The network is combinational.	1

Chapter 7

Indices

- genindex
- search